



SISTEMA DE RPG DE MESA

RGB System

Regras Centrais

V2.0 - PORTUGUESE (BRAZIL)
2026-08-28

MIT
256F364

Ficha tecnica

TITLE

RGB System - Regras Centrais

VERSION

v2.0 (2026-08-28)

LANGUAGE

Portuguese (Brazil) (pt-BR)

AUTHORS

RGB System contributors

LICENSE

MIT

SOURCE

<https://github.com/SergioLacerda/rpg-system-rgb> @ 256f364

GENERATOR

MkDocs + WeasyPrint

FUNDAMENTOS	6
Visão geral	7
Início rápido	10
Visão geral do sistema	14
Regras em uma página	17
SISTEMA BASE	21
Visão geral	22
Atributos	23
Criação de personagem	26
Progressão	29
Habilidades	32
COMBATE	37
Visão geral	38
Ataque e defesa	39
Modelo de decisão de combate	42
Modelo de dano	46
Movimento	50
EQUIPAMENTOS	54
Visão geral	55
Armaduras	56

■ Equipamentos gerais	59
■ Shields	62
■ ARMAS	65
■ Armas	66
■ REFERÊNCIA	67
■ Visão geral	68
■ Ficha de personagem	70
■ Exemplo de combate	73
■ Passo a passo de combate	77
■ Extra	81
■ Exemplo de jogo	85
■ Loop de jogo	89
■ Modelo de interação de dano	92
■ Architecture Notes	98
■ Engine do sistema	104
■ GLOSSÁRIO	110
■ Glossário	111
■ APÊNDICES	113
■ ADR-001: AI-First Documentation Authority	114
■ ADR-002: RGB Core V2 Design Control	116
■ ADR-003: Documentation L10n Maintenance Pattern	117
■ ADR-004: Modular Monolith In A Single Monorepo	119

ADR-005: Zero External Dependency Posture	121
ADR-006: cmd/rgb As The Unified CLI	124
ADR-007: Bundle Expansion Requires A Consumer Contract First	127
ADR-008: Retire The Go HTML/Print/PDF Publication Pipeline	130
ADR-009: Retain The MkDocs/Python Publication Exception Until Replacement Parity	132
ADR-010: Go-Owned Publication Without Python	135
ADR-011: Split License Model	137
ADR-012: MIT-Only License	138
ADR-013: Skill Distribution Via The Existing Publication Pipeline	140
ADR-014: RGB Specialist Bundled References Snapshot	143
ADR-015: Skill Runtime Split — LLM Instruction Layer vs. Go Component Layer	145
ADR-016: Reinstate A Narrow External PDF Authoring Exception	148
Architecture Decision Records	151
Explosivos	152
Armas de Fogo	155
Armas Corpo a Corpo	158
Penetração	160
Alcance de Armas	162

Fundamentos

NESTA SEÇÃO

7	Visão geral
10	Início rápido
14	Visão geral do sistema
17	Regras em uma página

Sistema RGB — Índice da Documentação (PT-BR)

Este índice central organiza toda a documentação do **Sistema RGB** em português.

O objetivo é facilitar a navegação entre os módulos principais do sistema.

RGB System

Sistema de RPG mantendo regras simples e combate tático.

O **Sistema RGB** utiliza três vetores:

Vetor	Nome	Função
R	Vermelho	pressão, impacto, disrupção e transformação da fonte de perigo
G	Verde	relação, mobilidade, timing, esquiva e posicionamento
B	Azul	preservação, escudos, estabilização, proteção e resistência

Foi projetado com quatro princípios:

- simplicidade estrutural
- profundidade tática
- modularidade
- escalabilidade

Ele permite adaptação para:

- campanhas modernas
- fantasia
- ficção científica
- superpoderes

Introdução

Documentos que apresentam os conceitos fundamentais do sistema.

- [Visão Geral do Sistema](#)

- [Início Rápido](#)
- [Regras em Uma Página](#)

Sistema Base

Define os fundamentos do sistema RGB.

- [Atributos](#)
- [Criação de Personagem](#)
- [Habilidades](#)

Combate

Regras que governam encontros de combate.

- [Movimento](#)
- [Ataque e Defesa](#)
- [Modelo de Dano](#)
- [Modelo de Decisão de Combate](#)

Equipamentos

Equipamentos que afetam sobrevivência e tática.

- [Armaduras](#)
- [Escudos](#)
- [Equipamentos Gerais](#)

Armas

Tipos de armas e regras relacionadas.

- [Armas de Fogo](#)
- [Armas Corpo a Corpo](#)
- [Explosivos](#)
- [Penetração](#)
- [Alcance de Armas](#)

Referência

Materiais auxiliares e exemplos práticos.

- [Ficha de Personagem](#)

- [Exemplo de Combate](#)
- [Exemplo de Jogo](#)
- [Loop de Jogo](#)
- [Engine do Sistema RGB](#)

Decisões de Arquitetura

ADRs do projeto são mantidos separadamente das regras do sistema.

- [ADR-001: Autoridade da Documentação AI-First](#)
- [ADR-002: Controle de Design do RGB Core V2](#)

Estrutura da Documentação

```
PT-br/  
├── introduction/  
├── core/  
├── combat/  
├── equipment/  
├── weapons/  
└── reference/
```

Cada pasta possui um **README.md próprio** que funciona como índice local da seção.

Sistema RGB – Início Rápido

Este guia de início rápido explica as ideias centrais do **Sistema RGB** e como começar a jogar em poucos minutos.

1. Os Vetores RGB

Os personagens no Sistema RGB são definidos por três vetores:

Vetor	Nome	Função
R	Vermelho	pressão, impacto, interrupção
G	Verde	relação, movimento, reação
B	Azul	preservação, escudos, estabilização

Esses três valores definem como um personagem atua em combate e interação.

2. Criando um Personagem

Um personagem inicial normalmente recebe:

7 pontos

Os jogadores distribuem esses pontos entre **R**, **G** e **B**.

Exemplo:

R = 3
G = 2
B = 2

3. Durabilidade do Personagem

A durabilidade é calculada usando fórmulas simples.

Vida = 4 + R + B
Escudo = B × 3

Exemplo:

R = 3
B = 2
Vida = 9
Escudo = 6

4. Estrutura Básica de Turno

Durante o combate um personagem normalmente tem:

Movimento (G × 2 metros, livre)
+
1 Ação
+
1 Ação Menor (opcional)

Exemplos:

Ações:

- atacar
- defender
- usar habilidade

Ações menores:

- recarregar arma
- mudar posição levemente
- interagir com o ambiente

5. Decisões de Combate

O combate gira em torno de três escolhas táticas:

Atacar
Mover
Defender

Essas ações correspondem diretamente aos vetores RGB:

Pressionar → R
Reposicionar → G
Sustentar → B

Os jogadores naturalmente enfatizam diferentes estratégias dependendo da distribuição de seus vetores.

6. Resolução de Dano

Quando um ataque acerta, o dano segue esta sequência:

Verificação de Acerto ou Contato
↓
Fonte de Impacto
↓
Penetração
↓
Redução por Armadura
↓
Absorção por Escudo
↓
Dano Restante → Personagem

Armadura reduz o dano **por ataque**, enquanto escudos absorvem **dano acumulado**.

7. Exemplo de Situação de Combate

Um personagem com:

R = 3
G = 2
B = 2

enfrenta um inimigo.

O jogador deve escolher entre:

- aumentar o dano causado
- reposicionar-se
- reforçar a defesa

Cada decisão corresponde a um dos vetores RGB.

8. Loop de Jogo

Uma sessão típica de RGB segue esta estrutura:

Criação de Personagem
↓
Exploração
↓
Encontro de Combate
↓
Recuperação
↓
Progressão da História

Esse ciclo se repete conforme a história avança.

Saiba Mais

Para uma compreensão mais profunda veja:

- Regras Básicas → [Regras Básicas](#)
- Sistema de Combate → [Sistema de Combate](#)
- Equipamentos → [Equipamentos](#)
- Armas → [Armas](#)
- Loop de Jogo → [Loop de Jogo](#)

Visão Geral do Sistema

O Sistema RGB é construído em torno de três vetores fundamentais:

- **R (Vermelho)** — poder, ataques e dano
- **G (Verde)** — agilidade, mobilidade e reação
- **B (Azul)** — escudos, energia e resistência especial

Esses vetores definem como os personagens interagem com o mundo e com o combate.

Estrutura do Sistema

O sistema é dividido em várias seções principais.

Regras Básicas

As regras básicas definem os vetores RGB e a criação de personagens.

- [Atributos](#)
- [Criação de Personagem](#)
- [Habilidades](#)

Sistema de Combate

As regras de combate definem como os personagens interagem durante encontros.

- [Ataque e Defesa](#)
- [Modelo de Decisão de Combate](#)
- [Modelo de Dano](#)
- [Movimento](#)

Equipamentos

Equipamentos fornecem opções defensivas e táticas.

- [Armaduras](#)
- [Escudos](#)
- [Equipamentos](#)

Armas

Armas determinam como o dano é aplicado durante o combate.

- [Armas de Fogo](#)
- [Armas Corpo a Corpo](#)
- [Explosivos](#)

Referência

Exemplos e materiais de apoio para o jogo.

- [Ficha de Personagem](#)
- [Exemplo de Combate](#)
- [Exemplo de Jogo](#)

Fluxo de Jogo

Uma interação típica no jogo segue esta estrutura:

Criação de Personagem

- Seleção de Equipamentos
- Posicionamento Tático
- Resolução de Combate
- Recuperação e Progressão

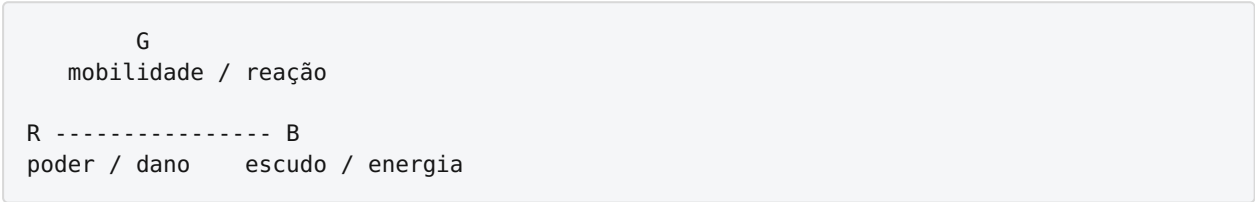
Notas Adicionais de Design

Explicações técnicas adicionais do sistema RGB podem ser encontradas aqui:

- [Arquitetura do Sistema RGB](#)

Modelo de Interação RGB

O sistema pode ser visualizado como um triângulo:



Cada vetor representa uma estratégia diferente de interação no combate.

Sistema RGB — Regras em Uma Página

Uma referência mínima para começar a jogar o **Sistema de RPG RGB** imediatamente.

Conceito Central

Os personagens são definidos por três vetores:

Vetor	Nome	Função
R	Vermelho	pressão, impacto, disrupção
G	Verde	relação, mobilidade, reação
B	Azul	preservação, escudos, estabilização

Os jogadores distribuem **7 pontos iniciais** entre R, G e B.

Exemplo:

R = 3

G = 2

B = 2

Durabilidade do Personagem

$$\begin{aligned}\text{Vida} &= 4 + R + B \\ \text{Escudo} &= B \times 3\end{aligned}$$

Vida representa endurance física mais preservação.

Escudo representa energia ou proteção especial.

Estrutura de Turno

A cada turno um personagem tem:

Movimento ($G \times 2$ metros, livre)
+
1 Ação
+
1 Ação Menor

Exemplos:

Ações:

- atacar
- defender
- usar habilidade

Ações menores:

- recarregar
- pequeno reposicionamento
- interagir com objeto

Escolhas Táticas

A maioria das decisões de combate se divide em três categorias:

Pressionar → R
Reposicionar → G
Sustentar → B

Isso cria três estratégias principais de combate:

Estratégia Vetor
----- -----
Atacante R
Escaramuçador G
Guardião B

Construções híbridas também são possíveis.

Resolução de Ataque

Um ataque é bem-sucedido quando o atacante supera o defensor de acordo com as regras de combate.

Se o ataque for bem-sucedido, o dano é resolvido.

Fluxo de Dano

```
Verificação de Acerto ou Contato
↓
Fonte de Impacto
↓
Penetração
↓
Redução por Armadura
↓
Absorção por Escudo
↓
Dano Restante → Personagem
```

Definições:

Armadura → reduz dano por ataque

Escudo → absorve dano acumulado

Exemplo de Dano

Fonte de Impacto: 7

Penetração: 2

Armadura: 4

```
Armadura Efetiva = Armadura - Penetração
Armadura Efetiva = 2
Dano Final = 7 - 2 = 5
```

Se houver escudo, o dano é absorvido pelo escudo primeiro.

Loop de Jogo

Uma sessão típica segue esta estrutura:

```
Criação de Personagem
↓
Exploração
↓
Combate
↓
Recuperação
↓
Progressão da História
```

Modelo Tático RGB

G
mobilidade / reação

R - B
poder / dano escudo / defesa

R → causar dano ou mudar a fonte de pressão

G → evitar dano ou mudar a relação com a pressão

B → absorver dano ou preservar continuidade sob pressão

Filosofia de Design

O sistema RGB foca em:

- relações numéricas simples
- decisões táticas de combate
- regras modulares
- cenários adaptáveis

As mesmas regras podem suportar:

- campanhas modernas
- mundos de fantasia
- ficção científica
- cenários com superpoderes

Sistema base

NESTA SEÇÃO

22	Visão geral
23	Atributos
26	Criação de personagem
29	Progressão
32	Habilidades

Sistema Base

Esta seção contém as regras fundamentais do **Sistema RGB**.

Aqui são definidos os vetores principais, a criação de personagem e as habilidades disponíveis.

Tópicos

Atributos

Define os três vetores RGB e suas funções no sistema.

→ [Atributos](#)

Criação de Personagem

Regras para criação de personagens utilizando os vetores RGB.

→ [Criação de Personagem](#)

Progressão

Regras de avanço de personagem: crescimento de vetor e escolhas alternativas de avanço.

→ [Progressão](#)

Habilidades

Sistema opcional de habilidades e capacidades especiais.

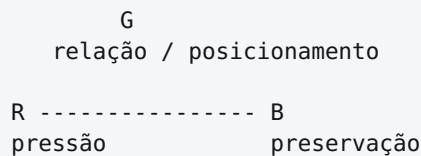
→ [Habilidades](#)

← [Voltar para README](#)

Atributos

O Sistema RGB é construído em torno de três vetores fundamentais que definem como personagens interagem com o mundo e com o combate.

Esses vetores formam o **Triângulo Tático RGB**, no qual cada atributo representa uma abordagem estratégica diferente.



- **R (Vermelho)** muda a fonte de pressão por força, impacto, disrupção ou transformação.
- **G (Verde)** muda a relação do personagem com a pressão por movimento, timing, evasão, alcance ou posicionamento.
- **B (Azul)** preserva continuidade sob pressão por endurance, bloqueio, escudos, estabilização, resistência ou proteção.

Juntos, esses vetores criam a dinâmica tática central do Sistema RGB.

Vetores RGB

Vermelho (R)

Representa **pressão e impacto físico**.

Usos típicos:

- força física;
- pressão de ataque;
- força de impacto;
- interrupção;
- quebrar, forçar ou transformar a fonte de perigo.

Personagens com **R** alto tendem a favorecer estratégias agressivas de combate.

Verde (G)

Representa **relação, timing e posicionamento**.

Usos típicos:

- mobilidade;
- reflexos;
- evasão;
- reposicionamento tático;
- alcance, timing, ocultação, detecção e controle de contato.

Personagens com **G** alto se destacam em controlar distância e evitar ataques.

Azul (B)

Representa **preservação sob pressão**.

Usos típicos:

- endurance;
- controle de energia;
- bloqueio e estabilização;
- escudos mágicos ou tecnológicos;
- resistência a dano;
- proteção de aliados ou posições.

Personagens com **B** alto se especializam em sobrevivência e táticas defensivas.

Interpretação Tática

Os três vetores correspondem às principais decisões de combate no sistema RGB:

```
Pressionar → R
Reposicionar → G
Sustentar → B
```

Essa relação forma a base da filosofia de combate do RGB. Jogadores naturalmente enfatizam estratégias diferentes dependendo da distribuição de seus vetores.

Exemplos:

Estilo de Personagem	Foco de Vetor
Atacante pesado	R alto
Combatente móvel	G alto
Guardião defensivo	B alto

Builds híbridas também são possíveis combinando vetores.

Escala de Poder

A escala abaixo fornece uma interpretação geral dos valores de vetor.

Valor	Interpretação
1	fraco
3	humano treinado
5	profissional
7	elite
9	excepcional
10	extraordinário

Vetores podem ultrapassar **10** por habilidades, equipamentos ou poderes especiais, dependendo do cenário da campanha.

Filosofia de Design

O modelo de atributos RGB mantém a criação de personagens simples enquanto suporta profundidade tática.

Cada vetor representa uma filosofia de combate diferente:

R → causar dano ou mudar a fonte de pressão
G → evitar dano ou mudar a relação com a pressão
B → absorver dano ou preservar continuidade sob pressão

Como cada personagem precisa distribuir seus pontos entre esses vetores, o sistema naturalmente produz builds diversas.

Veja também:

- [Criação de Personagem](#)

← [Voltar para README](#)

Criação de Personagem

A criação de personagens no **Sistema RGB** é intencionalmente simples. Jogadores distribuem pontos entre os três vetores RGB para definir como o personagem atua em combate e interação.

G
relação / posicionamento

R ----- B
pressão preservação

- **R (Vermelho)** → pressão, força, ataque, interrupção e impacto
- **G (Verde)** → movimento, timing, evasão, alcance e posicionamento
- **B (Azul)** → endurance, bloqueio, estabilização, escudos e proteção

Essa distribuição define o estilo tático do personagem.

Pontos Iniciais

Personagens começam com:

7 pontos

Esses pontos devem ser distribuídos entre:

R (Vermelho)
G (Verde)
B (Azul)

Exemplo:

R = 3
G = 2
B = 2

Diferentes distribuições naturalmente criam diferentes papéis em combate.

```
| Estilo | Foco de Vetor |
|-----|-----|
Atacante pesado | R alto |
Combatente móvel | G alto |
Guardião defensivo | B alto |
```

Builds híbridas também são possíveis.

Progressão do Personagem

À medida que personagens ganham experiência, eles se tornam mais fortes. Veja [Progressão](#) para as regras completas de avanço, incluindo a escolha por avanço entre crescimento de vetor e alternativas como novas habilidades, especialização ou novos recursos.

Vida

A durabilidade do personagem usa tolerância à pressão e preservação.

```
Vida = 4 + R + B
```

Exemplo:

```
R = 3
B = 2
Vida = 9
```

Isso impede que **R** possua sozinho ataque e durabilidade. **R** maior ainda ajuda personagens a suportar pressão física, enquanto **B** maior representa preservação, estabilidade e endurance defensiva.

Referência Rápida

```
Ataque      : margem = R atacante - G defensor
Movimento   : G × 2 metros
Vida        : 4 + R + B
Escudo      : B × 3
```

Essas fórmulas resumem as principais interações mecânicas do sistema RGB.

Filosofia de Design

A criação de personagens no RGB segue três princípios:

- **simplicidade** — poucos números definem o personagem

- **diversidade tática** — diferentes distribuições de vetores criam estilos diferentes de jogo
- **modularidade** — habilidades e equipamentos podem expandir o sistema sem alterar suas regras centrais

Como todos os personagens distribuem pontos entre os mesmos três vetores, o sistema naturalmente produz builds equilibradas e variadas.

Veja também:

- [Atributos](#)
- [Progressão](#)
- [Combate](#)
- [Movimento](#)

← [Voltar para README](#)

Progressão

À medida que personagens ganham experiência, eles se tornam mais fortes. A progressão no **Sistema RGB** é construída sobre um orçamento de avanço fixo e pequeno por nível, que o jogador pode gastar como crescimento bruto de vetor ou, quando a campanha usa o módulo de Habilidades e Skills, como uma entre várias escolhas alternativas de avanço.

Este documento é a fonte canônica para progressão de personagem. Ele substitui o resumo que antes ficava embutido em [Criação de Personagem](#), que agora aponta para cá.

Orçamento de Avanço

+2 pontos de vetor por nível

O Mestre determina quando os personagens sobem de nível, de acordo com a campanha. Essa base nunca muda — cada escolha de avanço abaixo gasta o mesmo orçamento fixo, não o soma.

Escolhas de Avanço

Por padrão, o orçamento completo se torna pontos de vetor. Quando a campanha permite o módulo [Habilidades e Skills](#), o jogador pode gastar o avanço de um nível em uma alternativa abaixo. Vale exatamente uma escolha por nível; as escolhas não se acumulam dentro do mesmo nível.

ESCOLHA

EFEITO

Crescimento de vetor de +2 pontos, distribuídos entre R, G e B como o jogador escolher (padrão)

Nova habilidade Ganha uma habilidade cujo **tier** e **requirements** (ver o Contrato de Habilidade em [Habilidades e Skills](#)) o personagem atualmente cumpre

Melhoria de habilidade Reduz o **cost** de uma habilidade conhecida, estende sua **duration**, ou aumenta o teto de **limits** em um degrau, a critério do Mestre

Especialização Compromete-se com a árvore de habilidades de um vetor (Vermelho, Verde ou Azul): futuras escolhas de nova habilidade nessa árvore ignoram um tier de **requirements**

Novo recurso Ganha um pool de recurso definido pela campanha (ex.: uma carga extra de reação, um buffer de recuperação de callback) dimensionado pelo Mestre

Nova reação Ganha acesso a um procedimento de reação não disponível no tier atual do personagem

Acesso a novo estado ou manobra Ganha a capacidade de declarar um estado tático ou manobra (ver [Ataque e Defesa](#), [Movimento](#)) que antes era restrito ou indisponível

Especialização Na Prática

Especialização é a única escolha com efeito estrutural duradouro: ela compromete um personagem com a identidade de um vetor em vez de conceder um benefício imediato. Um personagem especializado em Vermelho não ganha uma habilidade Vermelha só por se especializar — a especialização muda como futuras habilidades Vermelhas são liberadas, conforme o campo **requirements** do Contrato de Habilidade. Isso mantém a especialização significativa sem exigir rastreamento de recursos além do que o Contrato de Habilidade já define.

Justificativa de Design

A análise inicial do RGB System V2 (intake **base_project**, §7.5) deixou essa escolha em aberto: crescimento numérico puro, ou uma escolha por avanço entre várias opções. Este documento resolve isso como um **híbrido**, não uma invenção nova:

- A base numérica pura (**+2 pontos de vetor por nível**) já era canônica, declarada em Criação de Personagem antes deste documento existir. Este documento mantém isso inalterado como padrão — nenhum exemplo ou regra existente que assumia crescimento numérico puro é invalidado.
- [Habilidades e Skills](#) já mostrava habilidades liberadas por nível em seus exemplos trabalhados (**Nível 3 → +1 R para ataques à distância**, **Nível 6 → Golpe de poder duplo**, **Nível 9 → Ataque de impacto em área**) — uma escolha de habilidade por avanço já estava implícita na prática, só nunca formalizada como opção documentada ao lado do crescimento de vetor. Este documento apenas torna esse padrão existente explícito e consistente com os campos **tier**/

requirements do Contrato de Habilidade, que já existem especificamente para liberar acesso a habilidades por avanço.

- Um híbrido mantém intacta a filosofia de design declarada do sistema: criação de personagem e progressão favorecem **simplicidade** (um orçamento fixo, uma escolha por nível) e ao mesmo tempo suportam **diversidade tática** (a campanha, via o módulo opcional de Habilidades e Skills, pode oferecer mais que números puros sem adicionar novos subsistemas).

Exemplo

Um personagem de nível 6 com **R=5, G=3, B=4** e o módulo de Habilidades e Skills ativo pode escolher:

```
Crescimento de vetor → R=7, G=3, B=4 (ou qualquer divisão de +2)
Nova habilidade       → ganha uma habilidade com tier <= tier atual do personagem
Especialização       → compromete-se com Vermelho, Verde ou Azul para futura liberação
de habilidades
```

O jogador escolhe uma; a escolha não acumula nem carrega para o próximo nível.

Veja também:

- [Criação de Personagem](#)
- [Habilidades e Skills](#)
- [Atributos](#)

← [Voltar para README](#)

Habilidades

Habilidades representam capacidades especiais além das ações normais do Sistema RGB. Elas permitem que personagens expandam suas capacidades dependendo do cenário da campanha.

Habilidades são **módulos opcionais** e podem representar:

- técnicas marciais
- poderes sobrenaturais
- tecnologia avançada
- sistemas de magia

O Mestre determina se essas habilidades estão disponíveis na campanha.

Estrutura de Habilidades RGB

As habilidades no Sistema RGB são organizadas em torno dos **três vetores centrais**.

```
      G
    mobilidade / posicionamento

R ----- B
poder / dano      escudo / defesa
```

Cada vetor forma naturalmente uma **árvore de habilidades**.

- **R (Vermelho)** → habilidades de pressão, impacto, interrupção e transformação
- **G (Verde)** → habilidades de movimento, timing, evasão, alcance e posicionamento
- **B (Azul)** → habilidades de preservação, estabilização, bloqueio, escudo e resistência

Essa estrutura permite que o sistema escale naturalmente com diferentes cenários.

Árvore de Habilidades Vermelha (R)

Habilidades vermelhas ampliam **pressão ofensiva e força física**.

Habilidades típicas:

- Dobrar Potência

- Ataque Retardado
- Ataque de Pressão de Ar
- Golpe de Poder
- Quebra de Armadura

Essas habilidades aumentam a capacidade de um personagem causar impacto.

Exemplo de progressão:

Nível 3 → +1 R em ataques à distância (Precisão)
Nível 6 → Golpe de poder duplo
Nível 9 → Ataque de impacto em área

Árvore de Habilidades Verde (G)

Habilidades verdes focam em **mobilidade, velocidade e posicionamento tático**.

Habilidades típicas:

- Reflexos Ampliados
- Aceleração Momentânea
- Passo Fantasma (movimento curto similar a teleporte)
- Esquiva Automática
- Corrida Sobrenatural
- Salto Ampliado
- Movimento Silencioso
- Distorção de Movimento
- Evasão Sobrenatural

Essas habilidades permitem controlar espaço e evitar dano.

Árvore de Habilidades Azul (B)

Habilidades azuis representam **manipulação de energia, endurance e sistemas defensivos**.

Habilidades típicas:

- Escudo de Energia
- Escudo Atordoante
- Escudo Inteligente / Armadura Viva
- Invisibilidade
- Clones
- Absorção de Energia

Essas habilidades melhoram sobrevivência, continuidade e defesas baseadas em energia.

Contrato de Habilidade

Toda habilidade deve declarar os campos abaixo antes de poder ser tratada como canônica, validada, empacotada em bundles ou usada por um Specialist:

CAMPO	PROPÓSITO
<code>id</code>	identificador estável
<code>name</code>	nome usado em mesa
<code>vector</code>	vetor RGB primário
<code>tier</code>	nível, rank ou tier de acesso
<code>requirements</code>	pré-requisitos
<code>action_type</code>	ação, reação, passiva, postura ou timing especial
<code>cost</code>	pontos de vetor, escudo, callback, carga de item ou outro custo
<code>range</code>	alcance do alvo ou área
<code>duration</code>	instantânea, rodada, cena, persistente ou condicional
<code>effect</code>	efeito mecânico
<code>limits</code>	por turno, por combate, cooldown ou restrição narrativa
<code>tags</code>	tags de classificação
<code>source_status</code>	proposta, opcional, canônica, depreciada ou exemplo

Habilidades sem esses campos são exemplos ou notas de design, não regras canônicas.

Fator Marcial (Módulo Opcional)

O **Fator Marcial** introduz técnicas defensivas avançadas.

Exemplos:

- Defesa Completa
- Absorção Especial
- técnicas de postura defensiva

O Mestre decide se essas técnicas exigem:

- uma ação
- uma reação

- uma postura defensiva

Fator Azul (Módulo Opcional)

Em campanhas que permitem **magia ou meta-habilidades**, o vetor Azul pode manipular defesas baseadas em energia.

Essas habilidades normalmente interagem com o sistema de escudo:

$$\text{Escudo} = B \times 3$$

Exemplos:

- escudos mágicos
- barreiras de energia
- absorção de energia
- campos defensivos avançados

Sistema de Callback (Opcional)

Se a campanha inclui habilidades poderosas, o Mestre pode aplicar um **sistema de callback** para representar custo ou fadiga causada por essas habilidades.

Se callback ultrapassar **100%**, o personagem desmaia.

Intensidade	Recuperação
leve	horas
moderada	dias
grave	semanas
extrema	meses

O Mestre também pode aplicar consequências permanentes dependendo da situação.

Absorção Especial

Absorção Especial permite que personagens reduzam dano recebido usando um recurso declarado ou expressão vetorial.

1 ponto de vetor gasto → reduz 1 dano

Usos possíveis:

- **Vermelho (R)** → absorção de impacto físico
- **Azul (B)** → absorção de energia ou dano especial

O Mestre pode ajustar essa proporção dependendo da campanha.

Defesa Completa

Defesa Completa é um nome legado para bloqueio ou postura de guarda em que o personagem se prepara para receber pressão em vez de esquivar.

Essa postura também anula tentativas de agarrar.

Defesa Completa = procedimento de Bloqueio + efeito de absorção declarado

Categorias de Habilidades de Exemplo

Mobilidade

- voo
- levitação
- teleporte

Percepção

- visão noturna
- sensores avançados
- percepção ampliada

Tecnologia

- controle remoto de dispositivos
- habilidades de hacking
- interface com drones

Criando Habilidades RGB

Toda habilidade deve usar o contrato acima. No mínimo, ela deve definir vetor, custo, tipo de ação, efeito, duração, limites e status de origem.

Isso mantém habilidades consistentes com o Sistema RGB.

Veja também:

- [Criação de Personagem](#)
- [Atributos](#)

← [Voltar para README](#)

Combate

NESTA SEÇÃO

38	Visão geral
39	Ataque e defesa
42	Modelo de decisão de combate
46	Modelo de dano
50	Movimento

Combate

Esta seção descreve como os encontros de combate funcionam no **Sistema RGB**.

O combate envolve movimentação tática, resolução de ataques e cálculo de dano.

Tópicos

Movimento

Regras para deslocamento, posicionamento e interação com o terreno.

→ [Movimento](#)

Ataque e Defesa

Define como ataques são resolvidos entre personagens.

→ [Ataque e Defesa](#)

Modelo de Dano

Explica como o dano percorre as camadas do sistema.

→ [Modelo de Dano](#)

Modelo de Decisão de Combate

Mostra como os vetores RGB influenciam decisões táticas.

→ [Modelo de Decisão de Combate](#)

← [Voltar para README](#)

Ataque e Defesa

Iniciativa

Mais pontos em Verde (G) → age primeiro. Ataque surpresa ignora iniciativa.

Filosofia de Resolução

Core V2 resolve ataques com uma **margem**. Isso mantém o jogo determinístico possível enquanto permite resultados mais ricos do que acerto ou erro plano.

Margem de Ataque = Atacante (R + modificadores) - Defensor (G + modificadores)

Use a margem da seguinte forma:

MARGEM	RESULTADO
3 ou mais	acerto forte
1 a 2	acerto
0	acerto com custo ou abertura para resposta
-1 a -2	erro com oportunidade
-3 ou menos	erro claro

O atalho de ensino antigo continua válido:

$R \text{ do Atacante} \geq G \text{ do Defensor}$

Esse atalho significa que o atacante obtém ao menos um acerto básico quando a margem é zero ou maior.

Ataque

Pressão ofensiva. O atacante usa Vermelho (R) para mudar o alvo ou a fonte de perigo.

$\text{margem} = \text{Atacante (R + modificadores)} - \text{Alvo (G + modificadores)}$

Se o resultado for um acerto, continue para o modelo de dano ou para a consequência sem dano declarada.

Procedimentos Defensivos

Defesa não é mais um número combinado. Escolha o procedimento que descreve o que o defensor está fazendo.

PROCEDIMENTO	DONO PRIMÁRIO	PROPÓSITO
Esquivar	G	evitar ou alterar contato
Reposicionar	G	mudar alcance, cobertura ou engajamento
Bloquear	B ou equipamento	receber pressão intencionalmente
Redução por armadura	equipamento	reduzir impacto por acerto
Absorção por escudo	recurso derivado de B ou equipamento	absorver dano restante
Proteger aliado	B, equipamento ou habilidade	redirecionar ou conter pressão
Interromper	R	parar uma ação aplicando pressão primeiro
Contra-atacar	R ou G por procedimento	responder pressão por força ou timing

Não colapse esses procedimentos em uma soma genérica de armadura e escudo. Armadura e escudo agem depois na pipeline de dano.

Esquivar

Esquivar usa G contra o R do atacante.

Esquivar muda contato, timing, alcance ou posição. Se a esquivar for bem-sucedida, nenhum dano é aplicado, salvo se uma habilidade ou efeito de área disser o contrário.

Bloquear

Bloquear usa B, equipamento ou uma habilidade para receber pressão intencionalmente. Um bloqueio pode reduzir a margem de ataque, proteger outro alvo ou enviar o ataque para as camadas de armadura e escudo.

Agarrar

Permite imobilizar o oponente ou movê-lo forçadamente durante um turno.

A ação de agarrar tem êxito quando:

Atacante (G) $\geq$ Alvo (G)

Atacante (R) $\geq$ Alvo (R)

0 atacante pode usar meta-habilidades.

0 alvo pode usar bônus pré-declarados.

Dano

Se um ataque causa dano, use o modelo de dano.

Resolução de Dano

1. verificar acerto
2. estabelecer Fonte de Impacto
3. aplicar penetração
4. aplicar redução por armadura
5. aplicar absorção por escudo
6. aplicar dano restante à vida ou consequência de estado

Meta Habilidades ou Magia

Ataque mágico durante o combate

A cada turno, pontos são recuperados para usar magias ou meta-habilidades.

Regeneração = 1 ponto B por nível

Defesa mágica durante o combate

Defesa mágica é um escudo, bloqueio, resistência ou efeito de absorção tipado. Ela deve declarar qual procedimento defensivo modifica e se age antes ou depois da armadura.

Veja também:

- [Armas de Fogo](#)
- [Armaduras](#)

← [Voltar para README](#)

Modelo de Decisão de Combate

O Sistema RGB incentiva os jogadores a tomarem decisões táticas durante o combate.

Cada vetor RGB representa uma abordagem estratégica diferente.

Triângulo Tático RGB

As decisões de combate podem ser visualizadas usando o triângulo RGB.

G
mobilidade / posicionamento

R ----- B
poder / dano escudo / defesa

Cada vetor representa uma estratégia de combate diferente.

Estratégias de Combate

R – Estratégia Ofensiva

Foco em dano direto e combate agressivo.

Ações típicas:

- ataques poderosos
- armas pesadas
- quebrar defesas inimigas

Vantagens:

- alto dano
- eliminação rápida de inimigos

Fraqueza:

- menor sobrevivência

G – Estratégia de Mobilidade

Foco em movimento e posicionamento.

Ações típicas:

- flanquear inimigos
- evitar ataques
- controlar distância

Vantagens:

- mais difícil de atingir
- melhor posicionamento tático

Fraqueza:

- menor dano direto

B – Estratégia Defensiva

Foco em sobrevivência e proteção.

Ações típicas:

- uso de escudos
- habilidades defensivas
- absorção de ataques

Vantagens:

- alta durabilidade
- grande capacidade defensiva

Fraqueza:

- resolução de combate mais lenta

Interações Táticas

O sistema RGB permite múltiplos estilos de combate.

```
| Estratégia | Foco do Vetor |
|-----|-----|
Lutador agressivo | Alto R |
Escaramuçador móvel | Alto G |
Guardião defensivo | Alto B |
```

Construções híbridas também são possíveis.

Exemplos:

```
| Build | Estilo |
|-----|-----|
R + G | atacante rápido |
R + B | guerreiro pesado |
G + B | defensor evasivo |
```

Fluxo de Decisão de Combate

Durante o combate um jogador geralmente escolhe entre três opções táticas:

```
Pressionar
Reposicionar
Sustentar
```

Essas ações se relacionam naturalmente com os vetores RGB.

```
Pressionar → R
Reposicionar → G
Sustentar → B
```

Interação com o Modelo de Dano

As decisões de combate interagem diretamente com o sistema de dano.

```
Ataque
↓
Fonte de Impacto
↓
Penetração
↓
Redução por Armadura
↓
Absorção por Escudo
↓
Personagem
```

O jogador deve escolher entre:

- mudar a fonte de pressão
- mudar a relação com a pressão
- preservar continuidade sob pressão

Filosofia de Design

O Sistema RGB cria um modelo tático simples:

R → causar dano ou mudar a fonte de pressão
G → evitar dano ou mudar a relação com a pressão
B → absorver dano ou preservar continuidade

Essas três estratégias formam o núcleo das decisões táticas em combate no RGB.

Modelo de Dano

O Sistema RGB usa um **modelo de dano em camadas** que combina fontes de impacto, penetração, armaduras, escudos e habilidades especiais. Esse modelo mantém o combate simples enquanto permite decisões táticas.

Este documento explica como o dano flui pelo sistema.

Fluxo de Resolução de Dano

O dano no Sistema RGB segue uma sequência clara:

```
Verificação de Acerto ou Contato
↓
Fonte de Impacto
↓
Penetração
↓
Redução por Armadura
↓
Absorção por Escudo
↓
Dano Restante → Personagem
```

Esse fluxo mostra como diferentes elementos do sistema interagem durante o combate.

Resolução Passo a Passo

1. Verificação de Ataque

O atacante tenta acertar o defensor.

As regras de combate determinam se o ataque é bem-sucedido.

Veja:

- [Ataque e Defesa](#)

Se o ataque falhar, nenhum dano é aplicado.

2. Fonte de Impacto

Se o ataque for bem-sucedido, identifique a **Fonte de Impacto**.

Impacto pode vir de uma arma, atributo, habilidade, procedimento ou exceção explícita. O padrão é:

- impacto corpo a corpo usa o valor da arma mais o **R** do atacante quando a arma ou procedimento permitir;
- impacto de arma de fogo usa o valor da arma por padrão;
- impacto explosivo usa o perfil do explosivo;
- impacto de habilidade usa o efeito declarado pela habilidade.

Exemplos não podem sobrescrever a Fonte de Impacto declarada.

Veja:

- [Armas de Fogo](#)
- [Armas Corpo a Corpo](#)
- [Explosivos](#)

3. Penetração

Aplique penetração à armadura antes de a armadura reduzir dano.

$$\text{Armadura Efetiva} = \text{Armadura} - \text{Penetração}$$

Se a armadura efetiva for zero ou menor, a armadura não reduz aquele acerto.

Veja:

- [Penetração](#)

4. Redução por Armadura

Armadura fornece **proteção física**.

A armadura reduz dano recebido por acerto depois da penetração.

Categorias típicas de armadura incluem:

- Armadura Leve
- Armadura Média
- Armadura Pesada

Veja:

- [Armadura](#)

5. Absorção por Escudo

Alguns personagens podem possuir escudos.

Escudos absorvem dano restante **depois da redução por armadura** e antes que o dano alcance a vida ou gere consequência de estado.

Escudos de energia normalmente são calculados como:

$$\text{Escudo} = B \times 3$$

Veja:

- [Escudos](#)

6. Dano Restante

Depois que armadura e escudos são resolvidos, o dano restante é aplicado ao personagem.

Esse dano afeta a vida do personagem ou cria uma consequência de estado declarada.

Habilidades Especiais

Algumas habilidades podem modificar como o dano é aplicado.

Exemplos:

- técnicas de absorção
- escudos de energia
- técnicas marciais defensivas

Veja:

- [Habilidades](#)

Visão Geral da Interação de Dano

O modelo de dano RGB pode ser resumido como:

```
Fonte de Impacto
  ↓
Penetração
  ↓
Redução por Armadura
  ↓
Absorção por Escudo
  ↓
Dano Restante
```


Esse modelo em camadas mantém o combate fácil de entender enquanto permite que equipamentos e habilidades diferentes interajam de modo significativo.

Modelo de Interação RGB

O sistema também pode ser visualizado como uma interação entre os vetores RGB:

```
      G
    mobilidade / reação

R - B
poder / dano    escudo / energia
```

Cada vetor representa uma estratégia defensiva ou ofensiva diferente no combate.

- **R (Vermelho)** muda a fonte de pressão por força, impacto ou interrupção.
- **G (Verde)** muda a relação com a pressão por movimento, timing e posicionamento.
- **B (Azul)** preserva continuidade por bloqueios, escudos, estabilização e resistência.

Juntos, eles criam um sistema tático equilibrado.

← [Voltar para README](#)

Movimento

O movimento no **Sistema RGB** é governado principalmente pelo **vetor Verde (G)**, que representa agilidade, mobilidade e velocidade de reação.

As regras de movimento se conectam diretamente ao **Triângulo Tático RGB**, onde cada vetor representa uma estratégia diferente de combate.

G
mobilidade / posicionamento

R ----- B
poder / dano escudo / defesa

- **R (Vermelho)** foca em dano e poder ofensivo
- **G (Verde)** foca em mobilidade e posicionamento
- **B (Azul)** foca em defesa e resistência

O movimento, portanto, representa o **componente de vantagem posicional** no sistema RGB.

Distância de Movimento

A distância que um personagem pode se mover durante um turno de combate é:

Movimento = $G \times 2$ metros

Exemplo:

$G = 3$
Movimento = 6 metros por turno

Valores mais altos de **G** permitem que personagens se reposicionem mais rapidamente e controlem melhor o campo de batalha.

Grade Tática

O Sistema RGB suporta combate tático usando uma grade quadrada.

Grade padrão:

1 quadrado = 1 metro

Isso permite posicionamento preciso para:

- cobertura
- flanqueamento
- linha de visão
- efeitos de área

Movimento Aéreo

Se um personagem puder voar, o movimento aéreo utiliza a mesma fórmula:

Movimento Aéreo = $G \times 2$ metros

O Mestre pode modificar esse valor dependendo do cenário.

Exemplos incluem:

- jetpacks
- voo mágico
- superpoderes

Estrutura do Turno de Combate

Cada turno de combate normalmente inclui:

- **Movimento** — um deslocamento livre ($G \times 2$ metros); não consome a Ação nem a Ação Menor abaixo.
- **Ação**
- **Ação Menor**

Exemplos de **Ações**:

- atacar
- defender
- usar habilidade

Exemplos de **Ações Menores**:

- recarregar arma
- pequeno reposicionamento
- interagir com o ambiente

Efeitos de Terreno

O terreno pode reduzir o movimento.

Terreno	Efeito
-----	-----
Leve	-1 movimento
Moderado	movimento ÷2
Difícil	movimento ÷3

Penalidades de terreno são aplicadas **após o cálculo do movimento base**.

Exemplo:

G = 4
Movimento Base = 8 metros

Terreno moderado → 4 metros

Regra Opcional: Corrida

Personagens podem realizar uma corrida para se mover mais rapidamente.

Corrida = $G \times 3$ metros

Restrições:

- Não é possível atacar no mesmo turno
- Apenas movimento e ações menores são permitidas

Essa regra permite que personagens com alta mobilidade se reposicionem rapidamente.

Interpretação Tática

Os vetores RGB influenciam as decisões em combate:

Ataque → R
Mover → G
Defender → B

Isso cria três estratégias distintas de combate:

Estratégia	Vetor
-----	-----
Atacante pesado	R
Combatente móvel	G
Guardião defensivo	B

O movimento representa, portanto, o **controle tático do espaço** no sistema de combate RGB.

← [Voltar para README](#)

Equipamentos

NESTA SEÇÃO

55 Visão geral

56 Armaduras

59 Equipamentos gerais

62 Shields

Equipamentos

Esta seção descreve os equipamentos utilizados pelos personagens.

Equipamentos influenciam sobrevivência, mobilidade e opções táticas durante o combate.

Tópicos

Armaduras

Define os tipos de armadura e sua proteção física.

→ [Armaduras](#)

Escudos

Escudos físicos e energéticos que absorvem dano.

→ [Escudos](#)

Equipamentos Gerais

Itens diversos que personagens podem carregar ou utilizar.

→ [Equipamentos](#)

← [Voltar para README](#)

Armadura

Armaduras representam **proteção física** usada para reduzir dano recebido no Sistema RGB.

A armadura interage com o **Modelo de Dano** e pode coexistir com escudos, mas ela não se soma a escudos para formar uma defesa total genérica. Armadura é uma camada de redução por acerto; escudo é uma camada posterior de absorção.

Proteção de Armadura

A proteção da armadura é aplicada como redução por acerto depois da penetração.

```
Armadura Efetiva = Armadura - Penetração  
Dano Após Armadura = Fonte de Impacto - Armadura Efetiva
```

Se a Armadura Efetiva for zero ou menor, a armadura não reduz aquele acerto.

Exemplo:

```
Fonte de Impacto = 7  
Armadura = 4  
Penetração = 1  
Armadura Efetiva = 3  
Dano após armadura = 4
```

A absorção por escudo, se existir, acontece depois dessa etapa.

Tipos de Armadura

Tipo	Proteção	Penalidade de Mobilidade
-----		-----
Leve	2	-1 G
Média	4	-2 G
Pesada	6	-3 G

Armadura Leve

- Proteção mínima

- Baixa penalidade de mobilidade
- Adequada para personagens ágeis

Armadura Média

- Proteção equilibrada
- Redução moderada de mobilidade

Armadura Pesada

- Alta proteção física
- Penalidade significativa de mobilidade

Usuários de armadura pesada dependem mais de procedimentos de bloqueio, posicionamento e preservação do que de evasão pura.

Interação da Armadura com o Sistema RGB

A armadura interage com os vetores RGB da seguinte forma:

R → define ou contribui para a Fonte de Impacto quando a ação permite
G → é afetado pelas penalidades de mobilidade da armadura
B → preserva continuidade por bloqueios, escudos e estabilização

Essa relação mantém o equilíbrio tático do RGB:

R → mudar a fonte de pressão
G → mudar a relação com a pressão
B → preservar continuidade sob pressão

Interação com o Motor de Dano

A armadura reduz dano **após a aplicação da penetração**.

Fluxo simplificado de dano:

Fonte de Impacto
↓
Penetração
↓
Redução por Armadura
↓
Absorção por Escudo
↓
Dano Restante → Personagem

Para as regras completas veja:

- [Modelo de Dano](#)

Filosofia de Design

Armaduras no Sistema RGB seguem três princípios:

- **valores simples** — fáceis de calcular durante o combate
- **trocas táticas** — maior proteção reduz mobilidade
- **compatibilidade com escudos** — permite camadas defensivas sem colapsar procedimentos diferentes

Isso garante que armaduras permaneçam relevantes sem tornar personagens excessivamente resistentes.

Veja Também

- [Escudos](#)
- [Modelo de Dano](#)

← [Voltar para README](#)

Equipamentos

Equipamentos representam **ferramentas, dispositivos e itens táticos** que personagens podem carregar ou usar durante o jogo.

Diferente de armas ou armaduras, equipamentos gerais normalmente **não modificam diretamente os cálculos de dano**, mas podem fornecer vantagens táticas durante exploração, investigação ou combate.

Categorias de Equipamentos

No Sistema RGB, equipamentos podem ser agrupados em três categorias principais.

Ferramentas

Itens usados para interagir com o ambiente ou resolver problemas.

Exemplos:

- gazuas
- kits de reparo
- kits médicos
- equipamento de escalada

Esses itens normalmente permitem que personagens realizem ações que de outra forma seriam impossíveis.

Dispositivos

Equipamentos tecnológicos ou especializados que fornecem vantagens funcionais.

Exemplos:

- scanners
- dispositivos de comunicação
- ferramentas de vigilância
- drones

Dispositivos podem conceder bônus situacionais ou novas habilidades dependendo do cenário da campanha.

Equipamento Tático

Itens projetados para fornecer vantagens durante encontros.

Exemplos:

- lanternas
- granadas de fumaça
- sinalizadores
- ferramentas de arrombamento

Esses itens frequentemente influenciam **posicionamento, visibilidade ou controle do campo de batalha**.

Interação com o Sistema RGB

Equipamentos podem interagir com os vetores RGB dependendo do item.

Exemplos:

R → ferramentas que aumentam força ou impacto
G → ferramentas de mobilidade ou dispositivos de posicionamento
B → dispositivos energéticos ou equipamentos de proteção

No entanto, a maioria dos equipamentos gerais **não modifica diretamente os valores dos vetores principais**, a menos que o Mestre determine o contrário.

Filosofia de Design

Equipamentos no sistema RGB seguem três princípios de design:

- **utilidade acima da complexidade** – equipamentos devem permitir ações em vez de introduzir mecânicas pesadas
- **flexibilidade tática** – itens devem permitir soluções criativas
- **adaptabilidade ao cenário** – equipamentos podem variar dependendo do mundo da campanha

Por exemplo:

- cenários modernos podem incluir eletrônicos e equipamentos táticos
- cenários de fantasia podem incluir ferramentas mágicas
- cenários de ficção científica podem incluir tecnologia avançada

Relação com Outros Sistemas

Equipamentos interagem com vários módulos do RGB:

Equipamentos → apoiam ações de gameplay
Armas → causam dano
Armaduras → reduzem dano
Escudos → absorvem dano

Devido a essa separação, os equipamentos permanecem **simples e modulares** dentro do sistema RGB.

Veja Também

- [Armadura](#)
- [Escudos](#)
- [Armas de Fogo](#)

← [Voltar para README](#)

Escudos

Escudos são **equipamentos ou recursos defensivos adicionais** que ajudam um personagem a preservar continuidade sob pressão no Sistema RGB.

Eles funcionam com **Armaduras** e o **Modelo de Dano**, criando camadas de defesa. Escudos não se somam a armadura para formar uma defesa total genérica; eles absorvem dano restante depois da redução por armadura, salvo regra específica em contrário.

Existem dois tipos principais de escudos:

- Escudos Físicos
- Escudos Energéticos / Mágicos

Escudos Físicos

Escudos físicos representam equipamentos defensivos carregados por um personagem.

Eles permitem bloquear, proteger aliados ou absorver dano restante, mas podem impor penalidades de mobilidade.

Escudos físicos podem ser **destruídos, derrubados ou perdidos** dependendo da decisão do Mestre.

Tipo de Escudo	Proteção	Penalidade de Mobilidade
-----	-----	-----
Escudo Leve	+1	0
Escudo Médio	+2	-1 G
Escudo Pesado	+3	-2 G

Características

- Apoia procedimentos de bloqueio ou proteção
- Pode afetar mobilidade (G)
- Sujeito a dano ou perda de equipamento

Escudos físicos são mais comuns em:

- cenários medievais
- ambientes de combate tático
- combate em curta distância

Escudos Energéticos ou Mágicos

Escudos energéticos ou mágicos representam **campos de energia protetores**, tecnologia avançada ou barreiras mágicas.

Diferente da armadura, esses escudos **absorvem dano em vez de reduzi-lo por acerto**.

Características:

- não regeneram durante o combate, salvo regra específica
- possuem valor fixo durante um confronto
- retornam ao valor máximo após o término do combate quando o cenário permite

O valor do escudo é determinado pelo vetor Azul.

$$\text{Escudo} = B \times 3$$

Exemplo:

$$\begin{aligned} B &= 3 \\ \text{Escudo} &= 9 \end{aligned}$$

Escudos energéticos representam a **camada de absorção** no sistema de dano RGB.

Interação com o Modelo de Dano RGB

Os escudos são aplicados **após a redução de dano pela armadura**.

Ordem de resolução do dano:

```
Fonte de Impacto
↓
Penetração
↓
Redução por Armadura
↓
Absorção por Escudo
↓
Dano Restante → Personagem
```

Esse sistema de defesa em camadas mantém o combate simples enquanto preserva profundidade tática.

Interação com os Vetores RGB

Escudos interagem com os três vetores RGB de maneiras diferentes.

R → pode definir a fonte de pressão que o escudo tenta conter
G → pode ser reduzido por escudos pesados
B → determina a capacidade de escudos energéticos e preservação

Isso mantém a filosofia de combate do RGB:

R → mudar a fonte de pressão
G → mudar a relação com a pressão
B → preservar continuidade sob pressão

Filosofia de Design

Escudos no Sistema RGB seguem três princípios:

- **defesa em camadas** — armadura reduz dano, escudos absorvem dano
- **trocas táticas** — escudos mais pesados reduzem mobilidade
- **design modular** — escudos se adaptam a diferentes cenários

Exemplos:

- campanhas medievais → escudos físicos
- ficção científica → escudos energéticos
- cenários de fantasia → escudos mágicos

Veja Também

- [Armadura](#)
- [Modelo de Dano](#)
- [Ataque e Defesa](#)

← [Voltar para README](#)

CAPÍTULO 05

Armas

NESTA SEÇÃO

66 Armas

Armas

As armas definem os principais meios de causar dano no Sistema RGB.

O sistema separa **categorias de armas** de **mecânicas de armas**, permitindo que as regras sejam reutilizadas e mantidas simples.

Categorias de Armas

Estas páginas descrevem os diferentes tipos de armas disponíveis no sistema.

- [Armas de Fogo](#)
- [Armas Corpo a Corpo](#)
- [Explosivos](#)

Mecânicas de Armas

Estas regras são utilizadas por várias armas e definem como elas funcionam no combate.

- [Alcance de Armas](#)
- [Penetração](#)

Veja Também

Outras partes do sistema que interagem diretamente com armas:

- [Modelo de Dano](#)
- [Armadura](#)
- [Ataque e Defesa](#)

← [Voltar para o Índice](#)

Referência

NESTA SEÇÃO

68	Visão geral
70	Ficha de personagem
73	Exemplo de combate
77	Passo a passo de combate
81	Extra
85	Exemplo de jogo

Referência

Esta seção contém materiais de referência e exemplos práticos do **Sistema RGB**.

Esses documentos ajudam jogadores e mestres a compreender como as regras funcionam na prática.

Exemplos

Ficha de Personagem

Modelo de ficha padrão para personagens RGB.

→ [Ficha de Personagem](#)

Exemplo de Combate

Exemplo de combate utilizando as regras do sistema.

→ [Exemplo de Combate](#) → [Combate Walkthrough](#)

Exemplo de Jogo

Exemplo narrativo de uso do sistema durante uma sessão.

→ [Exemplo de Jogo](#)

Loop de Jogo

Explica o fluxo típico de uma sessão RGB.

→ [Loop de Jogo](#)

Notas de Design

Core V2

Documentos de controle do modelo Core V2.

Os registros de design Core V2 foram consolidados no pacote de análise inicial e ainda não possuem páginas publicadas nesta árvore de documentação.

Modelo de Interação RGB

Explica a relação entre vetores, dano e combate.

→ [Modelo de Interação RGB](#)

Arquitetura do Sistema

Observações sobre a arquitetura do sistema RGB.

→ [Arquitetura do Sistema](#)

Notas Extras

Observações adicionais sobre o design do sistema.

→ [Notas Extras](#)

Lacunas Conhecidas / Notas de Manutenção

- **Verificação de links:** esta documentação atualmente é verificada manualmente. Recomenda-se adicionar um verificador automático de links markdown ao CI para evitar que referências quebradas passem despercebidas.
- **RGB Ability Engine:** uma referência anterior a um documento `rgb_ability_engine.md` foi removida de `rgb_system_engine.md` — nenhum documento desse tipo chegou a ser criado neste repositório.

← [Voltar para README](#)

Ficha de Personagem

A ficha de personagem resume os **valores principais de um personagem no Sistema RGB**.

Ela organiza os três vetores RGB junto com valores derivados e equipamentos.

Informações Básicas

Nome
Nível

Vetores RGB

R (Vermelho) → Pressão / Impacto
G (Verde) → Relação / Movimento / Reação
B (Azul) → Preservação / Escudos / Estabilização

Esses vetores determinam a maioria das ações no sistema.

Valores Derivados

Valores derivados são calculados a partir dos vetores RGB.

$Vida = 4 + R + B$

Escudo energético opcional (se o cenário permitir):

$Escudo\ Energético = B \times 3$

Sistema opcional de fadiga de habilidades:

Callback (se utilizado no cenário da campanha)

Equipamentos

Equipamentos
Armas
Armadura

Os equipamentos interagem com os sistemas de combate e dano.

Relação com o Sistema RGB

A ficha de personagem conecta diversos módulos do sistema RGB.

Vetores → definem as capacidades do personagem
Vida → derivada de R + B
Escudo → derivado de B
Equipamentos → modificam a interação em combate

Filosofia de Design

A ficha de personagem RGB segue três princípios:

- **estrutura mínima** – poucos números definem o personagem
- **leitura rápida** – todos os valores principais visíveis rapidamente
- **modularidade do sistema** – sistemas opcionais (escudo, callback) podem ser adicionados conforme a campanha

Isso permite que a mesma ficha funcione em:

- campanhas modernas
- cenários de fantasia
- mundos de ficção científica
- jogos com superpoderes

Exemplo Mínimo de Ficha

Nome: Stormy

Nível: 3

R: 3

G: 3

B: 1

Vida: 8

Escudo: 3

Armadura: Armadura Média

Armas: Pistolas Duplas

Equipamentos: Equipamento Tático

← [Voltar para README](#)

Exemplo de Combate

Este exemplo demonstra como o combate é resolvido usando o **Sistema RGB**.

Dois personagens entram em combate e resolvem ações usando as regras principais:

- vetores RGB
- movimento
- ataque e defesa
- armadura e escudos
- cálculo de dano

Personagens

Personagem A – Combatente de Assalto

R = 3
G = 2
B = 1

Valores derivados:

Vida = $4 + R + B = 8$
Escudo = $B \times 3 = 3$
Movimento = $G \times 2 = 4$ metros

Equipamento:

- Armadura Média (Proteção 4)
- Rifle

Personagem B – Operador Móvel

R = 2
G = 3
B = 2

Valores derivados:

Vida = 8
Escudo = 6
Movimento = 6 metros

Equipamento:

- Armadura Leve (Proteção 2)
- SMG

Exemplo de Turno de Combate

Passo 1 — Posicionamento

Os personagens se movem de acordo com:

Movimento = $G \times 2$ metros

O Personagem B se move primeiro devido à maior mobilidade.

Passo 2 — Ataque

O Personagem A dispara o rifle.

Exemplo de Fonte de Impacto:

Fonte de Impacto = 6
Penetração = 2

Passo 3 — Interação com Armadura

A armadura reduz o dano recebido após a penetração.

Exemplo:

Armadura do Alvo = 2
Penetração = 2
Armadura Efetiva = 0

A armadura é ignorada.

Passo 4 — Absorção por Escudo

O escudo absorve dano antes da vida.

Escudo = 6
Dano restante = 6

Resultado:

Escudo Restante = 0
Dano à Vida = 0
Vida Restante = 8

Segundo Ataque

O Personagem B ataca com uma SMG.

Fonte de Impacto = 4
Penetração = 1

Passo 5 — Interação com Armadura

Armadura do Alvo = 4
Penetração = 1
Armadura Efetiva = 3

Dano = 4
Armadura Efetiva = 3
Dano Restante = $4 - 3 = 1$

Passo 6 — Absorção por Escudo

O Personagem A ainda tem o escudo cheio (3), então o dano restante atinge o escudo antes da vida.

Escudo = 3
Dano Restante = 1

Resultado:

Escudo Restante = $3 - 1 = 2$
Dano à Vida = 0
Vida Restante = 8

Resumo do Fluxo de Dano

O sistema RGB resolve dano usando uma estrutura em camadas.

```
Fonte de Impacto
↓
Penetração
↓
Redução por Armadura
↓
Absorção por Escudo
↓
Dano Restante → Personagem
```

Interpretação Tática

O exemplo mostra o papel dos vetores RGB:

```
R → muda a fonte de pressão
G → muda a relação com a pressão
B → preserva continuidade sob pressão
```

Cada vetor sustenta uma estratégia de combate diferente.

Objetivo de Design

A resolução de combate no sistema RGB foi projetada para ser:

- rápida de calcular
- taticamente significativa
- consistente entre cenários

Como as mesmas fórmulas se aplicam a todos os personagens, o sistema permanece fácil de balancear.

Veja Também

- [Ataque e Defesa](#)
- [Movimento](#)
- [Modelo de Dano](#)

← [Voltar para README](#)

Exemplo Completo de Combate

Este exemplo demonstra um **combate completo** utilizando o Sistema RGB.

Dois personagens entram em combate dentro de um ambiente tático.

Personagens

Stormy

R = 3

G = 3

B = 2

Valores derivados:

Vida = $4 + R + B = 9$

Escudo = $B \times 3 = 6$

Movimento = $G \times 2 = 6$ metros

Equipamentos:

- Armadura Média
- Duas Pistolas

Agente de Segurança

R = 2

G = 2

B = 1

Valores derivados:

Vida = 7

Escudo = 3

Movimento = 4 metros

Equipamentos:

- Armadura Leve
- SMG

Situação Inicial

Os personagens começam a **6 metros de distância**.

Stormy possui cobertura parcial atrás de um veículo.

Turno 1

Movimento

Stormy se move 2 metros para melhorar a linha de visão.

Fórmula de movimento:

Movimento = $G \times 2$

Stormy poderia se mover até **6 metros**, mas escolhe apenas um reposicionamento menor.

Ataque

Stormy dispara uma das pistolas.

Fonte de Impacto:

Impacto = 4

Penetração = 1

Interação com Armadura

Armadura do alvo:

Armadura Leve = 2

Penetração = 1

Armadura Efetiva = 1

Dano restante após armadura:

$$4 - 1 = 3$$

Interação com Escudo

Escudo do alvo:

$$\text{Escudo} = 3$$

O escudo absorve todo o dano.

$$\text{Escudo Restante} = 0$$

$$\text{Dano na Vida} = 0$$

Turno 2

O agente de segurança contra-ataca.

Arma:

Rajada de SMG

$$\text{Fonte de Impacto} = 5$$

$$\text{Penetração} = 1$$

Armadura de Stormy:

$$\text{Armadura Média} = 4$$

$$\text{Armadura Efetiva} = 3$$

Cálculo do dano:

$$5 - 3 = 2$$

O escudo de Stormy absorve o dano.

$$\text{Escudo Restante} = 4$$

$$\text{Dano na Vida} = 0$$

Resumo do Combate

Este exemplo ilustra o modelo de dano do sistema RGB.

Fluxo de dano:

Fonte de Impacto

↓

Penetração

↓

Redução por Armadura

↓

Absorção por Escudo

↓

Personagem

Observação Tática

Stormy possui vantagem devido a:

- maior mobilidade (G)
- escudo mais forte (B)

O agente depende mais da Fonte de Impacto da arma.

Isso demonstra como **diferentes distribuições de RGB influenciam a estratégia de combate.**

← [Voltar para README](#)

Observação de Equilíbrio Matemático no Sistema RGB

Esta seção documenta uma propriedade de equilíbrio que surge ao cruzar as principais fórmulas do sistema RGB. O objetivo é explicar como os vetores R, G e B sustentam estratégias diferentes sem fazer um único vetor dominar ataque e durabilidade.

1. Fórmulas Fundamentais do RGB

As regras básicas do sistema estabelecem as seguintes relações:

Vida

$$\text{Vida} = 4 + R + B$$

Movimento

$$\text{Movimento} = G \times 2$$

Escudo

$$\text{Escudo} = B \times 3$$

Absorção Especial

Absorção física ou energética usa um recurso declarado pela habilidade ou procedimento. Ela não substitui a pipeline de dano.

2. Impacto de Cada Vetor no Combate

Cada vetor influencia o combate de maneira diferente.

	Vetor	Função
R	pressão, impacto e interrupção	
G	relação, mobilidade, evasão e posicionamento	
B	preservação, escudos, estabilização e proteção	

3. Escala de Resistência

Considerando um personagem com **5 pontos em cada vetor**:

Vida = $4 + 5 + 5 = 14$
Escudo = $5 \times 3 = 15$

Observação:

Escudo $\approx$ Vida

Isso significa que a proteção baseada em energia possui resistência semelhante à durabilidade física e de preservação, mas opera em outra camada.

4. Equilíbrio Entre os Vetores

Cada vetor contribui para sobrevivência de maneira diferente.

Personagem focado em R

- maior pressão ofensiva
- mais tolerância a pressão física quando combinado à Vida

Personagem focado em B

- Vida maior por preservação
- escudo energético mais forte
- melhor estabilização, bloqueio ou absorção quando declarados

Resultado:

Ambos podem alcançar resistência relevante, porém por mecânicas diferentes.

5. Papel do Vetor G

O vetor G não aumenta diretamente a resistência.

Em vez disso, ele melhora:

- evasão
- mobilidade
- timing
- posicionamento tático

Isso cria uma terceira forma de sobrevivência: evitar, alterar ou atrasar o contato com a pressão.

6. Multiplicadores e Funções dos Vetores

```
| Vetor | Expressão principal |
|-----|-----|
R | margem de ataque e pressão |
G | movimento x2 e relação com pressão |
B | Vida compartilhada e Escudo x3 |
```

Isso cria três formas distintas de defesa:

```
R → interromper ou encerrar a fonte de pressão
B → preservar continuidade e absorver pressão
G → evitar ou reposicionar a relação com a pressão
```

7. Arquétipos Naturais do Sistema

Sem criar classes formais, o sistema gera estilos naturais de personagem.

```
| Estilo | Vetor dominante |
|-----|-----|
Atacante de pressão | R alto |
Evasivo | G alto |
Guardião de preservação | B alto |
```

8. Builds Híbridas

Exemplo:

R = 6

B = 6

```
Vida = 4 + 6 + 6 = 16
Escudo = 6 × 3 = 18
```

Resistência aproximada:

16 + 18 = 34 pontos antes de consequências de estado

Isso demonstra que **combinações de vetores também são viáveis e competitivas**.

9. Triângulo de Equilíbrio do RGB

O sistema pode ser representado por um triângulo funcional:

G
evasão / mobilidade

R ----- B
pressão preservação

Cada vetor representa uma estratégia diferente para sobreviver ao combate.

10. Conclusão

O Sistema RGB apresenta um equilíbrio estrutural:

- R muda a fonte de pressão
- B preserva continuidade e sustenta escudos
- G reduz, altera ou evita contato com a pressão

Esse triângulo cria um sistema naturalmente balanceado sem exigir classes fixas ou regras adicionais complexas.

← [Voltar para README](#)

Exemplo de Gameplay

Este exemplo ilustra como **uma sessão típica funciona no Sistema RGB**.

O jogo normalmente alterna entre três fases principais:

- interação narrativa
- exploração
- combate

Essas fases formam o **ciclo principal de gameplay** usado na maioria das campanhas RGB.

Cenário de Exemplo

Um pequeno grupo de operativos está investigando atividades suspeitas em um distrito industrial.

O grupo é composto por:

- **Stormy** – especialista em combate móvel
- **Agente de Segurança** – operador tático treinado
- **Técnico de Suporte** – especialista em equipamentos e análise

O Mestre descreve o ambiente e os jogadores decidem como proceder.

Fase 1 — Narrativa

A sessão começa com interpretação de personagens e interação narrativa.

Exemplo:

A equipe recebe informações sobre carregamentos ilegais de tecnologia em um distrito de armazéns.

Os jogadores podem:

- questionar NPCs
- coletar informações
- planejar a abordagem
- negociar ou intimidar

Nenhuma regra mecânica rígida é necessária nesta fase, a menos que o Mestre solicite testes.

Fase 2 — Exploração

Os jogadores se movimentam pelo ambiente e interagem com objetos ou locais.

Exemplos:

- vasculhar edifícios
- contornar sistemas de segurança
- escanear o ambiente
- evitar patrulhas

O Mestre pode solicitar testes RGB quando apropriado.

Exemplo:

```
Teste de Agilidade → baseado em G
Teste de Força → baseado em R
Interação técnica / energética → baseada em B
```

A exploração incentiva o uso criativo de equipamentos e habilidades.

Fase 3 — Combate

Se ocorrer um conflito, o jogo entra na **fase de combate**.

O combate utiliza o sistema de combate RGB.

Sequência básica:

```
Movimento
↓
Ação (ataque / habilidade)
↓
Resolução de defesa
↓
Cálculo de dano
```

Exemplo de encontro:

Stormy encontra dois agentes hostis dentro do armazém.

Os jogadores utilizam:

- movimento (G)
- ataques (R)
- escudos ou habilidades (B)

O confronto é resolvido usando as regras de combate RGB.

Resolução da Sessão

Após o encontro, o jogo retorna para a fase narrativa.

Possíveis resultados:

- personagens se recuperam do combate
- jogadores investigam o local
- novos elementos da história surgem

Esse ciclo continua ao longo da sessão.

Ciclo de Gameplay RGB

A estrutura geral de gameplay pode ser resumida como:

```
Narrativa
↓
Exploração
↓
Combate
↓
Recuperação / Progresso da História
```

Esse ciclo permite que o sistema RGB suporte diferentes estilos de jogo mantendo regras simples.

Filosofia de Design

O sistema RGB separa **fluxo narrativo** de **mecânicas de combate**.

- fases narrativas permanecem flexíveis
- exploração incentiva criatividade dos jogadores
- combate utiliza regras táticas claras

Esse equilíbrio permite que o mesmo sistema funcione em:

- campanhas modernas
- mundos de fantasia
- cenários de ficção científica
- histórias com superpoderes

Veja Também

- [gameplay_loop.md](#)
- [combat_example.md](#)
- [combat_walkthrough.md](#)

← [Voltar para README](#)

Loop de Jogo

O Sistema RGB segue um loop de jogo simples que conecta criação de personagens, exploração e combate.

Esse ciclo descreve como o sistema é normalmente utilizado durante uma sessão.

1. Criação de Personagem

Os jogadores criam personagens usando o sistema de vetores RGB.

Os personagens distribuem pontos entre:

- **R (Vermelho)** — poder, ataques e força física
- **G (Verde)** — agilidade, mobilidade e reação
- **B (Azul)** — escudos, energia e resistência especial

Habilidades adicionais e equipamentos podem ser escolhidos dependendo da campanha.

→ Veja [Criação de Personagem](#)

2. Exploração e Interação

Durante o jogo, os personagens interagem com o ambiente usando seus atributos RGB.

Ações típicas incluem:

- investigação
- movimento
- interação social
- desafios do ambiente

O mestre determina como os atributos RGB influenciam essas ações.

3. Posicionamento Tático

Antes de o combate começar, os personagens se posicionam no ambiente.

O Sistema RGB suporta combate tático usando posicionamento em grade.

Considerações típicas incluem:

- distância entre personagens
- linha de visão
- cobertura e terreno
- velocidade de movimento

→ Veja [Movimento](#)

4. Resolução de Combate

O combate é resolvido usando as regras de combate do RGB.

As interações de combate envolvem:

- comparações de ataque e defesa
- dano de armas
- interação entre armadura e escudos
- movimento tático

→ Veja [Ataque e Defesa](#)

5. Dano e Consequências

O dano afeta os personagens através de:

- armadura
- escudos
- vitalidade

Habilidades especiais podem modificar essas interações.

→ Veja [Armas](#)

6. Recuperação

Após encontros, os personagens podem se recuperar dependendo das regras definidas pelo mestre.

A recuperação pode incluir:

- recuperação de vida
- recarga de habilidades
- reparo de equipamentos

Ciclo de Jogo

Uma sessão típica segue esta estrutura:

Criação de Personagem

- Exploração
- Posicionamento Tático
- Combate
- Consequências
- Recuperação

Esse ciclo se repete conforme a história avança.

← [Voltar para README](#)

Sistema RGB – Modelo de Dano e Notas de Interação

Este documento consolida observações de design sobre o modelo de dano do Sistema RGB, seu equilíbrio matemático e como os vetores RGB interagem com as mecânicas de combate.

O objetivo é explicar **por que o sistema se comporta de forma consistente e permanece equilibrado**, mantendo regras simples.

Modelo de Interação RGB

O Sistema RGB é construído em torno de três vetores fundamentais:

- **R (Vermelho)** — pressão, impacto e disrupção
- **G (Verde)** — relação, mobilidade e reação
- **B (Azul)** — preservação, escudos e estabilização

Esses vetores interagem para criar um combate tático.

G

mobilidade / reação

R - B

pressão preservação

Cada vetor influencia um aspecto diferente do jogo:

Vetor	Papel no Jogo
-----	-----
R	muda a fonte de pressão
G	muda a relação com a pressão
B	preserva continuidade sob pressão

Fluxo de Interação do Sistema

O Sistema RGB conecta atributos do personagem, equipamentos e mecânicas de combate.

```
Vetores RGB
↓
Criação de Personagem
↓
Seleção de Equipamentos
↓
Interação de Combate
↓
Resolução de Dano
```

Essa estrutura modular permite que o sistema se adapte a diferentes cenários:

- campanhas modernas
- mundos de fantasia
- cenários de ficção científica
- universos com superpoderes

Modelo de Dano RGB

O dano no combate segue uma estrutura em camadas.

```
Verificação de Acerto ou Contato
↓
Fonte de Impacto
↓
Penetração
↓
Redução por Armadura
↓
Absorção por Escudo
↓
Dano Restante → Personagem
```

Essa abordagem garante que fontes de impacto, armaduras e escudos interajam de forma previsível.

Fórmulas Principais do Sistema

O sistema RGB define durabilidade do personagem com fórmulas simples.

```
Vida = 4 + R + B
Escudo = B × 3
```

A Fonte de Impacto pode vir de arma, atributo, habilidade, procedimento ou exceção explícita. Armaduras oferecem **valores fixos de redução por acerto**.

A penetração modifica o valor efetivo da armadura.

Armadura Efetiva

A penetração interage com a armadura da seguinte forma:

$$\text{Armadura Efetiva} = \text{Armadura} - \text{Penetração}$$

O dano restante após armadura torna-se:

$$\text{Dano Restante} = \text{Fonte de Impacto} - \text{Armadura Efetiva}$$

Isso mantém os cálculos simples e previsíveis.

Escala Natural de Impacto

Se utilizarmos os seguintes intervalos:

Elemento	Escala Típica
-----	-----
Fontes de Impacto	3-10
Armadura	1-6
Penetração	0-4

Surge uma relação natural de equilíbrio:

$$\text{Fonte de Impacto} \approx \text{Armadura} + \text{Penetração}$$

Isso garante que ataques e defesas permaneçam relevantes.

Exemplo

Fonte Leve

Fonte de Impacto = 4
Penetração = 1
Armadura = 4

Armadura Efetiva = 4 - 1 = 3
Dano Restante = 4 - 3 = 1

Fonte Pesada

Fonte de Impacto = 8
Penetração = 3
Armadura = 6

Armadura Efetiva = $6 - 3 = 3$
Dano Restante = $8 - 3 = 5$

Interação com Escudos

Após a redução pela armadura, os escudos absorvem o dano restante.

Dano Restante – Escudo

Escudos seguem a fórmula RGB:

Escudo = $B \times 3$

Isso cria duas camadas defensivas:

Tipo de Defesa	Função
Armadura	reduz dano por acerto
Escudo	absorve dano acumulado restante

Essa separação simplifica a resolução do combate.

Regra Prática de Balanceamento

Ao projetar fontes de impacto e armaduras, a seguinte aproximação mantém o combate equilibrado:

Armadura $\approx$ Fonte de Impacto / 2
Penetração $\approx$ Fonte de Impacto / 3

Isso mantém a maioria dos ataques relevantes sem tornar as defesas inúteis.

Vantagem Inesperada: Combate Contra Múltiplos Inimigos

Uma propriedade interessante do modelo de dano RGB é que ele funciona bem quando personagens enfrentam **múltiplos oponentes simultaneamente**.

Muitos sistemas de RPG têm dificuldades nesse cenário porque valores defensivos não escalam bem contra ataques repetidos.

No RGB, o modelo em camadas distribui o dano naturalmente:

Fonte de Impacto → Penetração → Armadura → Escudo → Personagem

Armadura

A armadura reduz dano **em cada ataque individual**, evitando que inimigos fracos sobrecarreguem rapidamente personagens bem protegidos.

Escudo

Escudos absorvem **o dano total acumulado restante**, funcionando como um amortecedor contra vários ataques menores.

Consequências Táticas

Isso cria uma dinâmica de combate estável.

Tipo de Inimigo	Resultado
Muitos inimigos fracos	Armadura reduz a maioria dos ataques
Poucos inimigos fortes	Penetração torna-se mais importante
Inimigos com energia	Escudos tornam-se críticos

Como a armadura funciona por golpe e os escudos funcionam cumulativamente, o sistema permanece estável mesmo com muitos atacantes.

Conclusão de Design

A estrutura de dano do RGB forma uma hierarquia clara de defesa:

```
Fonte de Impacto
↓
Penetração
↓
Redução por Armadura
↓
Absorção por Escudo
↓
Personagem
```

Essa estrutura:

- mantém os cálculos simples

- suporta combate tático
- escala naturalmente com equipamentos
- lida bem com encontros com múltiplos inimigos

Arquitetura do Sistema RGB – Observação de Design em Camadas

Esta nota documenta uma propriedade estrutural interessante que aparece quando a documentação do RGB é analisada como um todo.

Embora o Sistema RGB tenha sido projetado como um sistema de RPG de mesa simples, a documentação revela naturalmente uma **arquitetura em camadas** semelhante à encontrada em engines de jogos.

Essa estrutura melhora a clareza, modularidade e capacidade de expansão a longo prazo.

Arquitetura em Camadas do Sistema RGB

Quando a documentação é organizada por função, o sistema forma as seguintes camadas:

```
Camada do Sistema
↓
Camada de Gameplay
↓
Camada de Combate
↓
Camada de Dano
↓
Camada de Equipamentos
```

Cada camada responde a uma pergunta diferente de design.

1. Camada do Sistema

Documentos:

- [Visão Geral do Sistema](#)
- [Modelo de Interação RGB](#)

Objetivo:

Esta camada explica a **filosofia central e a estrutura do sistema**.

Ela define os vetores RGB:

- **R (Vermelho)** – dano e poder físico
- **G (Verde)** – mobilidade e reação
- **B (Azul)** – escudos e defesa energética

Também mostra como os módulos do sistema se conectam.

Equivalente em engines de jogo:

Design do sistema central.

2. Camada de Gameplay

Documentos:

- [Loop de Jogo](#)
- [Modelo de Decisão de Combate](#)

Objetivo:

Esta camada explica **como os jogadores interagem com o sistema durante o jogo**.

Fluxo típico:

```
Criação de Personagem  
↓  
Exploração  
↓  
Combate  
↓  
Recuperação
```

Durante o combate o jogador normalmente escolhe entre:

```
Atacar  
Mover  
Defender
```

Que se relacionam naturalmente com:

```
Atacar → R  
Mover → G  
Defender → B
```

Equivalente em engines de jogo:

Estrutura de gameplay.

3. Camada de Combate

Documentos:

- [Ataque e Defesa](#)
- [Movimento](#)

Objetivo:

Define como os personagens interagem durante encontros.

Isso inclui:

- regras de movimento
- resolução de ataques
- reações defensivas
- posicionamento

Equivalente em engines de jogo:

Sistema de combate.

4. Camada de Dano

Documentos:

- [Modelo de Dano](#)
- [Modelo de Interação RGB](#)

Objetivo:

Responsável pela lógica interna do cálculo de dano.

O motor de dano RGB segue esta estrutura:

```
Fonte de Impacto
↓
Penetração
↓
Redução por Armadura
↓
Absorção por Escudo
↓
Personagem
```

Essa camada permite que o combate permaneça tático enquanto mantém os cálculos simples.

Equivalente em engines de jogo:

Motor de dano.

5. Camada de Equipamentos

Documentos:

- [Armaduras](#)
- [Escudos](#)
- [Armas de Fogo](#)
- [Armas Corpo a Corpo](#)
- [Explosivos](#)

Objetivo:

Define os **valores de dados utilizados pelos sistemas de combate e dano**.

Exemplos:

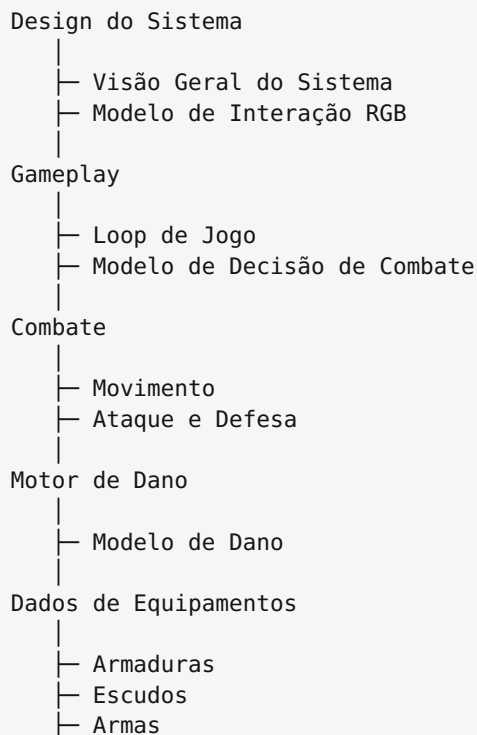
- dano de armas
- redução de armadura
- valores de penetração
- capacidade de escudos

Equivalente em engines de jogo:

Camada de dados.

Arquitetura Completa do Sistema RGB

Quando combinada, a documentação do RGB forma a seguinte estrutura:



Consequências de Design

Essa arquitetura em camadas produz diversas vantagens.

Modularidade

Novos subsistemas podem ser adicionados sem modificar as regras centrais.

Exemplos:

- sistemas de magia
- cibernética
- veículos
- superpoderes

Balanceamento Mais Simples

A maior parte do balanceamento ocorre em:

Camada de Dano
Camada de Equipamentos

O restante do sistema permanece estável.

Documentação Clara

Cada camada responde a uma pergunta diferente:

```
| Camada | Pergunta |  
|-----|-----|  
Sistema | Como o sistema é estruturado |  
Gameplay | Como os jogadores interagem com o jogo |  
Combate | Como os personagens interagem |  
Dano | Como o dano é calculado |  
Equipamentos | Quais valores numéricos existem |
```

Observação Final

O Sistema RGB se comporta menos como um conjunto tradicional de regras de RPG e mais como uma **engine reutilizável de sistema de RPG**.

Isso torna o sistema particularmente adequado para:

- campanhas modernas
- mundos de fantasia
- cenários de ficção científica
- universos com superpoderes

Nesse modelo:

```
Sistema RGB → engine de regras  
Cenário (ex.: Aurora) → mundo construído sobre a engine
```

Essa separação permite que o Sistema RGB permaneça genérico, enquanto diferentes cenários fornecem o contexto narrativo.

← [Voltar para README](#)

Engine do Sistema RGB

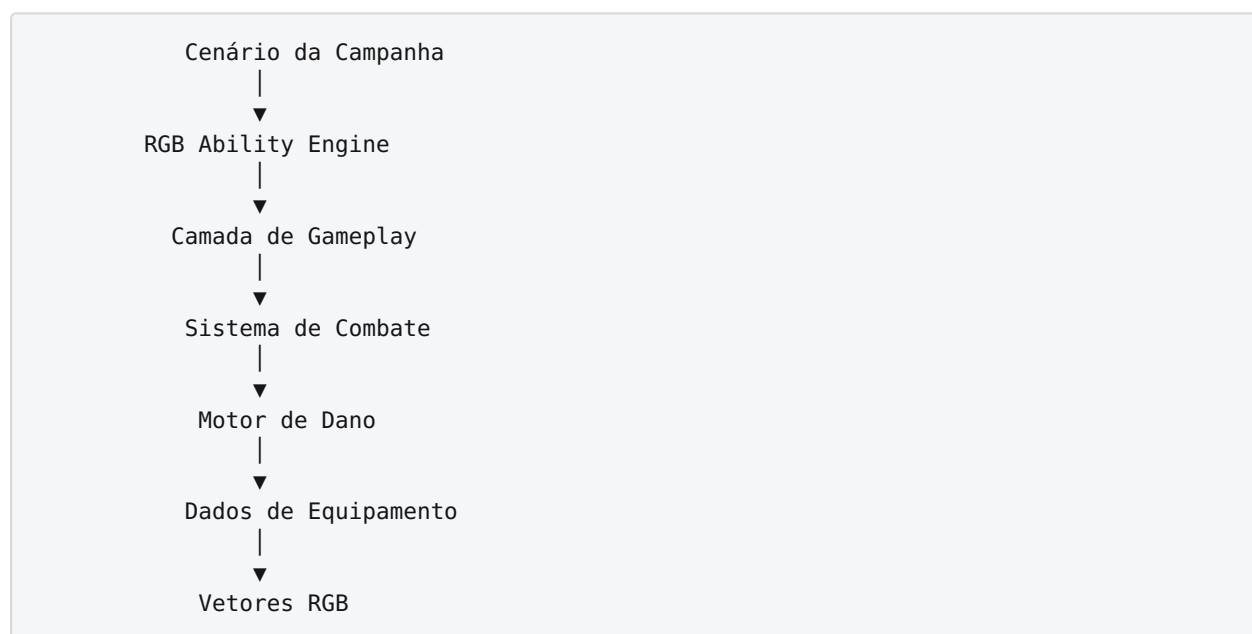
O **Engine do Sistema RGB** descreve a arquitetura completa do sistema de RPG RGB. Ele conecta todos os componentes do sistema em uma estrutura em camadas semelhante à de uma engine de jogos.

Esse modelo ajuda a explicar como o sistema RGB permanece:

- modular
- escalável
- fácil de balancear
- adaptável a diversos cenários

Arquitetura do Sistema RGB

O sistema RGB pode ser entendido como uma engine em camadas:



Cada camada se apoia na camada abaixo.

Descrição das Camadas

Vetores RGB (Camada Base)

Todo o sistema é construído sobre três vetores fundamentais.

G
relação / posicionamento

R ----- B
pressão preservação

Os vetores definem as mecânicas principais:

- **R (Vermelho)** → pressão, impacto e disrupção
- **G (Verde)** → relação, mobilidade e reação
- **B (Azul)** → preservação, escudos e estabilização

Todos os outros sistemas derivam desses três valores.

Dados de Equipamento

Essa camada define os valores numéricos utilizados pelo sistema de combate.

Exemplos:

- fontes de impacto
- redução de armadura
- valores de penetração
- capacidade de escudos

Documentos relacionados:

- [Armas de Fogo](#)
- [Armas Corpo a Corpo](#)
- [Armaduras](#)
- [Escudos](#)

Motor de Dano

O motor de dano determina como ataques afetam os personagens.

Fonte de Impacto
↓
Penetração
↓
Redução por Armadura
↓
Absorção por Escudo
↓
Personagem

Documentos relacionados:

- [Modelo de Dano](#)
- [Modelo de Interação RGB](#)

Sistema de Combate

A camada de combate define a interação entre personagens.

Exemplos:

- resolução de ataques
- movimento
- posicionamento
- reações defensivas

Documentos relacionados:

- [Movimento](#)
- [Ataque e Defesa](#)

Camada de Gameplay

Define como os jogadores interagem com o sistema durante o jogo.

Loop típico de jogo:

Criação de Personagem
↓
Exploração
↓
Combate
↓
Recuperação

Documentos relacionados:

- [Criação de Personagem](#)
- [Loop de Jogo](#)

- [Modelo de Decisão de Combate](#)

RGB Ability Engine

Essa camada introduz habilidades, poderes e mecânicas especiais.

As habilidades são construídas usando os vetores RGB.

```
id
name
vector
tier
requirements
action_type
cost
range
duration
effect
limits
tags
source_status
```

Documentos relacionados:

- [Habilidades e Perícias](#)

Cenário da Campanha

A camada superior define o mundo narrativo.

Exemplos:

- campanhas modernas
- mundos de fantasia
- ficção científica
- universos com superpoderes

Exemplo de separação:

```
Sistema RGB → engine de regras
Aurora → cenário construído sobre a engine
```

Fluxo Completo do Sistema

A interação completa entre as camadas:

```
Vetores
↓
Dados de Equipamento
↓
Motor de Dano
↓
Sistema de Combate
↓
Regras de Gameplay
↓
Engine de Habilidades
↓
Cenário da Campanha
```

Vantagens de Design

O RGB System Engine oferece diversas vantagens.

Design Modular

Novos subsistemas podem ser adicionados sem reescrever as regras centrais.

Exemplos:

- sistemas de magia
- veículos
- cibernética
- superpoderes

Balanceamento Mais Simples

A maior parte do balanceamento ocorre em:

```
Motor de Dano
Dados de Equipamento
```

O restante do sistema permanece estável.

Documentação Clara

Cada camada responde a uma pergunta específica.

Camada	Pergunta	
-----	-----Vetores	0 que define um persona-
gem?		
Equipamentos	Quais números existem?	
Motor de Dano	Como o dano funciona?	
Combate	Como personagens interagem?	
Gameplay	Como jogadores interagem?	
Habilidades	Quais poderes existem?	
Cenário	Em que mundo a história ocorre?	

Observação Final

O Sistema RGB se comporta menos como um conjunto tradicional de regras de RPG e mais como uma **engine modular de RPG**.

Graças a essa arquitetura, o mesmo sistema pode suportar diferentes gêneros mantendo simplicidade e consistência.

← [Voltar para README](#)

Glossário

NESTA SEÇÃO

111 Glossário

Glossário

Termos usados nas regras do Sistema RGB, reunidos em um só lugar. Esta página não é normativa por si só — cada entrada aponta para o documento de regra que de fato define o termo; o glossário só indica onde encontrar.

R, G, B

Os três vetores em que o sistema é construído. Veja [Atributos](#).

- **R (Vermelho)** — pressão e impacto físico: ataque, força, interrupção.
- **G (Verde)** — relação, timing e posicionamento: movimento, evasão, alcance.
- **B (Azul)** — preservação sob pressão: resistência, bloqueio, escudos, estabilização.

Margem

A diferença entre o **R** efetivo do atacante e o **G** efetivo do defensor em um ataque, usada para classificar o resultado (acerto forte, acerto, acerto com custo, erro com oportunidade, erro claro) em vez de um simples acerto/erro. Veja [Ataque e Defesa](#).

Vida

O recurso de vitalidade de um personagem, $Vida = 4 + R + B$. Veja [Criação de Personagem](#).

Fonte de Impacto

A força de um golpe antes da mitigação, antes de penetração, redução por armadura ou absorção por escudo serem aplicadas. Veja [Modelo de Dano](#).

Penetração

Quanto do impacto de uma arma ignora a armadura antes da redução por armadura ser aplicada. Veja [Penetração](#).

Redução por Armadura

A camada de mitigação que reduz o impacto por golpe, aplicada depois da penetração. Veja [Armadura](#).

Absorção por Escudo

A camada de mitigação que absorve o impacto restante depois da redução por armadura, drenando um recurso derivado de **B**. Veja [Escudos](#).

Estado

Uma condição marcada que um personagem pode carregar (por exemplo, **ferido**, **suprimido**, **oculto**), criada, alterada ou removida por procedimentos específicos. Existem quinze estados, divididos em categorias físicas, posicionais, táticas, defensivas e informacionais.

Procedimento

Uma ação defensiva ou tática com identidade estável e vetor definido: Esquivar, Reposicionar, Bloquear, Interromper, Contra-pressão. Veja [Ataque e Defesa](#).

Habilidade

Uma capacidade discreta de personagem declarada com um contrato mínimo governado (id, vetor, nível, requisitos, custo, alcance, duração, efeito, limites, tags, status de origem). Veja [Habilidades](#).

Nível

A faixa de poder/complexidade de uma habilidade, usada junto com os requisitos para definir quando um personagem pode adquiri-la. Veja [Habilidades](#).

Apêndices

NESTA SEÇÃO

114	ADR-001: AI-First Documentation Authority
116	ADR-002: RGB Core V2 Design Control
117	ADR-003: Documentation L10n Maintenance Pattern
119	ADR-004: Modular Monolith In A Single Monorepo
121	ADR-005: Zero External Dependency Posture
124	ADR-006: cmd/rgb As The Unified CLI

ADR-001: AI-First Documentation Authority

Status

Accepted.

Context

RGB Core V2 needs stable documentation identity before Maker, Specialist, tooling, Library, PDF, landing, bundles, AI context packs, or search indexes can consume rules safely.

The project uses Markdown as a human review surface, but AI-first retrieval, provenance, localized drift checks, projection manifests, and generation gates need stable source IDs and machine-readable contracts.

Decision

Adopt a bridge-first documentation authority model:

- English Markdown under `docs/core/en/**` remains the canonical bridge for Core V2 rule text until a specific semantic source unit is promoted.
- PT-br Markdown under `docs/core/PT-br/**` remains a localized projection and cannot introduce mechanics not present in canonical English or canonical semantic source units.
- `docs/core/semantic/core-v2.index.json` provides stable IDs, authority type, source status, projection paths, relationships, retrieval metadata, and provenance.
- Accepted semantic source files under `docs/core/semantic/source/**` may become canonical for specific promoted units when they preserve Markdown EN/PT-br projection paths, source IDs, provenance, and validation coverage.
- Generated artifacts, indexes, context packs, Library pages, PDF output, landing summaries, and bundles are derived projections only.

Consequences

- Consumers get stable IDs and provenance without waiting for a repository-wide rewrite.

- Tooling can validate source paths, projection paths, relationships, translations, consumer contracts, projection ownership, and generated output authority.
- Promoted semantic source files can replace Markdown as canonical rule authority for their specific source IDs.
- Generated artifacts remain non-canonical and must trace back to source IDs.

Rejected Alternatives

- Big-bang semantic rewrite.
- Markdown-only long-term source model.
- Generated artifacts as rule authority.

ADR-002: RGB Core V2 Design Control

Status

Accepted.

Context

RGB Core V2 needed a controlled design surface before downstream tracks such as Maker, Specialist, tooling, Library, PDF, landing, and bundles could consume the model safely.

Decision

Treat Core V2 as the authoritative source of the playable RGB model. Core owns:

- vector grammar;
- procedures;
- resources;
- tactical states;
- ability contract boundaries;
- documentation authority for rules, procedures, examples, and design notes.

Downstream tracks consume Core decisions. They do not define Core mechanics.

Consequences

- Core documentation can be normalized before downstream product surfaces.
- Maker and Specialist must consume stable Core IDs and contracts.
- Library, PDF, generated bundles, search indexes, and AI context packs remain derived surfaces.
- Documentation authority must stay explicit when rules are promoted into semantic source units.

ADR-003: Documentation L10n Maintenance Pattern

Status

Accepted.

Context

RGB System keeps English documentation as the canonical technical source and Portuguese documentation as a localized reader-facing surface. Path parity is useful, but it does not show whether a localized document is current, stale, or missing after an English source changes.

Core V2 documentation updates exposed this maintenance cost: English rule documents changed first, and PT-br alignment required a separate review pass.

Decision

Adopt a lightweight advisory L10n manifest for documentation maintenance.

The project path model is:

Canonical source	docs/core/en/**
Localized surface	docs/core/PT-br/**
Traceability layer	docs/core/semantic/l10n-manifest.v0.1.json

The manifest records:

- canonical English source path;
- localized path;
- target locale;
- authority type;
- translation status;
- source revision marker;
- optional notes.

English remains the canonical technical source. PT-br documents are localized reader-facing projections and must not introduce mechanics that are absent from the English source.

The first manifest is advisory. It supports review and tooling, but it does not generate localized documentation and does not block publication by itself.

Authority Types

- `concept`
- `rule`
- `procedure`
- `example`
- `design_note`
- `translation`
- `generated_artifact`

Translation Statuses

- `current`
- `needs_review`
- `stale`
- `missing`
- `intentionally_divergent`
- `not_applicable`

Review Workflow

1. Update the canonical English document.
2. Identify affected localized documents through the manifest.
3. Mark affected PT-br entries as `needs_review` or `stale`.
4. Update PT-br content in a separate documentation task.
5. Return the manifest entry to `current` only after review.

Consequences

- Maintainers can see source-to-localized traceability without reading every pair manually.
- Future Library, PDF, bundle, search, and Specialist consumers can disclose localization status before relying on localized content.
- Full semantic-source migration, per-document frontmatter, and automatic translation remain separate future decisions.

ADR-004: Modular Monolith In A Single Monorepo

Status

Proposed.

Context

RGB System V2 spans several planned products: Core rules, Maker, Specialist, Tooling, Library (HTML/PDF), and a landing page. The V2 initial analysis (base_project intake, section 20) lists two viable repository strategies: a monorepo or separate repositories per product.

The current state favors consolidation:

- the products share one canonical source (`docs/` Markdown + YAML) and one contract vocabulary that is still in formation;
- the Go scaffold already models products as component boundaries under `internal/components/`;
- there is a single maintainer and no divergent release cycles;
- validation must run across products (docs ↔ tooling ↔ skills), which is cheap in one repository and expensive across several.

Decision

Adopt a **modular monolith inside a single monorepo**.

- One Go module (`github.com/SergioLacerda/rpg-system-rgb`) with one binary entrypoint per product surface under `cmd/` as they materialize (`rgb` , later `rgb-compile` , `rgb-validate` , `rgb-query`).
- Each product is a component boundary under `internal/components/<name>` , isolated by enforced import rules (see below), so a future repository split is a directory move, not a rewrite.
- Non-Go surfaces stay as monorepo siblings: `docs/` (canonical content), `skills/` (lightweight Markdown/YAML skills), `web/` (Astro landing), `generated/` (derived artifacts only), `scripts/` (standalone validation module).

Boundaries are enforced mechanically, not by convention:

- `tests/architecture/` fails the build when a component imports `app`, `cmd`, or a sibling component;
- `.golangci.yml` depguard rules repeat the same constraints in the linter;
- `make review-structure` is the aggregate local gate (`docs/engineering/base-structure-review-workflow.md`).

Split Criteria

Extract a product into its own repository only when at least one of these holds (mirrors the V2 analysis, section 20.3):

- release cycles genuinely diverge;
- distinct teams or communities own different products;
- independent distribution produces real benefit (e.g. a public SDK);
- repository size becomes an operational obstacle.

Because component boundaries are import-clean by construction, a split requires moving `internal/components/<name>` plus its `cmd/` surface and publishing the shared contract package — nothing else should break.

Consequences

Positive:

- contracts between products stay synchronized in one place;
- cross-product validation (docs, schemas, bundles) runs in one gate;
- refactoring across boundaries is atomic;
- the architecture tests keep the monolith modular instead of letting it degrade into an entangled monolith.

Negative / accepted costs:

- one repository carries heterogeneous toolchains (Go, Node/Astro, Markdown pipelines);
- releases of independent products must be tagged within one history;
- contributors see the whole ecosystem even when touching one product.

ADR-005: Zero External Dependency Posture

Status

Accepted, with a documentation-PDF exception added on 2026-07-31.

Context

RGB System V2 has never imported anything outside the Go standard library, in any of its Go modules, at any point in its history — no `go.sum` exists anywhere in the repository. This was never decided in a single place; it emerged as a consistent pattern across independent missions, each reasoning locally that a proposed dependency would be "the first" of its kind and rejecting it on that basis:

- `docs-compiler-html-pdf-scope-20260726` rejected automating PDF generation via headless-Chromium (an external binary requirement) or a Go-native PDF library (a new Go module dependency), keeping PDF export a documented manual browser step instead.
- `guardrails-tooling-scope-20260726` rejected adopting `gocognit` (cognitive-complexity linting) specifically because it requires installing a new external Go tool, even though the project already had build-failing cyclomatic-complexity coverage via `gocyclo` (bundled with the already-required `golangci-lint` dev tool, not a module dependency).

Each decision was locally reasoned and undocumented as a standing rule. This ADR formalizes the pattern those decisions were already following, so future missions can cite one authoritative source instead of re-deriving the same reasoning independently each time.

Decision

RGB System V2's Go modules do not take on external dependencies — no `go.sum` entries, no external binaries, no runtimes beyond the Go toolchain itself — unless a future ADR explicitly revisits this decision.

This applies to:

- Go module dependencies (anything that would produce a `go.sum` entry);
- external binaries or runtimes invoked by any `make` target or `cmd/*` entrypoint (e.g. a headless browser, a Java runtime, a database server);

- build-time or render-time dependencies embedded into generated output (e.g. a JavaScript rendering library bundled into `generated/library/html/**`).

It does **not** apply to:

- developer tooling that is not a dependency of the module itself (`golangci-lint` is already required to run `make lint`, the same way a compiler is required to build; it does not appear in `go.sum` and does not ship inside any built artifact);
- plain-text, markup-embeddable formats with no rendering dependency of their own — see Consequences below for the Mermaid application of this principle.

Consequences

- Every future proposal to add a Go module, external binary, or runtime dependency must be weighed against this ADR, not re-argued from first principles — and requires a new ADR to override, per this repo's own `M016`-equivalent guardrail-non-regression pattern (see `tests/architecture/architecture_test.go`'s own header comment and `docs/engineering/base-structure-review-workflow.md`). `docs-readme-c4-adr-scope-20260726` applied this reasoning directly: C4 architecture diagrams (`docs/architecture/**`) use Mermaid syntax embedded in plain Markdown — no PlantUML (requires a Java runtime or a rendering server) and no Go diagrams-as-code library (a new module dependency) — specifically because both alternatives would violate this ADR.
- This keeps the project's build and review surface small: `go build ./...` and `go test ./...` never need network access or a lockfile audit.
- The project accepts the corresponding cost: some features (automated PDF generation, cognitive-complexity linting, richer diagram styling) stay deferred or manual rather than automated, until a future ADR explicitly decides the trade-off is worth it for a specific case.

Addendum 2026-07-31: Documentation PDF Generation Exception

Supersession note: this exception was retired for active publication by [ADR-010](#). The section below is kept as historical context for the temporary exception that existed before the Go-owned publication path replaced it.

The project accepts a narrow exception for automated PDF generation in the documentation publication pipeline.

This exception exists because the landing page and Library now expose stable PDF download links, and keeping those links fresh by manual browser export is operationally fragile. A generated PDF is a distribution artifact for readers, not a source of rules authority.

The exception allows:

- Python documentation-build dependencies required by a MkDocs PDF plugin;
- operating-system rendering libraries required by that plugin, when documented as setup prerequisites;
- Makefile targets dedicated to documentation PDF generation, such as `docs-pdf` or `pdf-build`;

- generated PDF files under the landing static download surface.

The exception does not allow:

- Go module dependencies or any `go.sum` entry;
- runtime dependencies used by game/system code;
- headless-browser automation unless a later ADR explicitly authorizes that class of dependency;
- treating generated PDFs as canonical documentation sources;
- silently making every build depend on PDF rendering before the dependency profile is documented and validated.

Implementation constraints:

- Keep the PDF dependency profile separate from the base documentation profile, for example `requirements-docs-pdf.txt` in addition to `requirements-docs.txt`.
- Prefer an explicit `docs-pdf` or `pdf-build` target during stabilization. `landing-build` may depend on it only after the PDF dependency profile is reproducible in the supported development environment.
- Generated PDFs must be traceable to canonical Markdown or semantic source units through the existing projection/manifest model.
- The landing page should link to stable `latest` aliases while preserving versioned PDF files for release history.
- Release manifest generation, checksum verification, and PDF editorial validation are project-owned tooling and should run through Go. Python remains allowed here only for the MkDocs renderer stack until a later ADR retires or replaces that publication path.

ADR-006: `cmd/rgb` As The Unified CLI

Status

Accepted. Amended 2026-08-02 — see [Amendment: publication pipeline retirement](#) and [Amendment: deprecated caller migration](#) below.

Context

A base project architecture review found the CLI surface ambiguous: `cmd/rgb` advertised scaffold readiness only (`app.Hello()`), while the functional commands were split across `cmd/rgb-tooling` (`validate`, `generate`, `bundle`) and `cmd/rgb-compiler` (`all`, `no-html`). Nothing distinguished which binary a user or CI job was supposed to run, and the review left the choice open: either promote `cmd/rgb` to a unified CLI, or keep it as a status command and document the other two as the real CLIs.

Decision

`cmd/rgb` is the unified CLI. It now dispatches to the same `internal/app` use cases the other two binaries already called:

```
rgb                -> scaffold/component status (unchanged)
rgb validate [repo-root] -> internal/app.ValidateDocs
rgb generate [repo-root] -> internal/app.GenerateProjections
rgb bundle  [repo-root] -> internal/app.BuildBundle
rgb compile  all|no-html [repo-root] -> internal/app.Compile
```

`Makefile` targets (`validate`, `generate`, `bundle`, `compile`, `compile-full`) now invoke `cmd/rgb` instead of the two split binaries.

`cmd/rgb-tooling` and `cmd/rgb-compiler` are **deprecated, not removed**. They still build and behave exactly as before because real callers depended on them at the time of this decision: `tests/semantic_docs/semantic_docs_test.go` shelled out to `go run ./cmd/rgb-tooling validate`, and `docs/core/semantic/README.md` documented `rgb-tooling` directly. Removing a working entrypoint those depended on was a breaking change under M006 (RFC Process for Breaking Changes) and was out of scope for the original decision.

Consequences

Positive:

- one documented entrypoint for the product-facing CLI surface, matching the "one coherent CLI" gap called out by the architecture critique;
- `internal/app` stays the single seam between entrypoints and components, so `cmd/rgb`'s dispatch logic is a thin, independently testable layer (`cmd/rgb/main_test.go`) with no new component coupling.

Negative / accepted costs:

- two deprecated binaries remain in the tree until their callers migrate;
- `cmd/rgb`'s subcommand set must be kept in sync with `internal/app` by hand until there's enough surface to justify a flag-parsing library (see ADR-005 on the zero external dependency posture for why that's not a default reach).

Follow-up

- Completed: `tests/semantic_docs/semantic_docs_test.go` and `docs/core/semantic/README.md` now call `cmd/rgb`.
- Remaining: propose `cmd/rgb-tooling` removal through the M006 RFC process if and when compatibility support is no longer needed. (`cmd/rgb-compiler` no longer applies here — see the publication-pipeline amendment.)

Amendment: Publication Pipeline Retirement (2026-08-02)

A base project Library review found that, unlike `cmd/rgb-tooling`, `cmd/rgb-compiler` and the `compile` subcommand had **no real caller** — no test, CI workflow, or required Makefile target used them; `internal/components/compiler` and `internal/components/library` underneath them were equally uncalled outside that path. M006's breaking-change concern (real callers depend on this) did not apply, so [ADR-008](#) removed `compile`, `cmd/rgb-compiler`, `internal/components/compiler`, and `internal/components/library` outright instead of deprecating them. Every mention of `compile`/`cmd/rgb-compiler` above describes this ADR's original decision at the time it was written, not the current subcommand set — see ADR-008 for what replaced it. The `cmd/rgb-tooling` decision above is unaffected.

Amendment: Deprecated Caller Migration (2026-08-02)

The repository callers named in the original decision have migrated to `cmd/rgb`: `tests/semantic_docs/semantic_docs_test.go` runs `go run ./cmd/rgb validate`, and `docs/core/semantic/README.md` documents `cmd/rgb` as the equivalent command for `make validate` and `make generate`.

`cmd/rgb-tooling` remains present only as a compatibility path. Removing it is still a breaking-change decision and requires the M006 RFC process.

ADR-007: Bundle Expansion Requires A Consumer Contract First

Status

Accepted.

Context

`internal/components/bundles` currently projects a minimal, real subset of the semantic index (`id`, `kind`, `title`, `source_status`, `relationships`) into `generated/bundle/rgb.bundle.json`. A base project architecture review confirms this is a useful minimum but flags that the plan's full bundle shape (`rules`, `procedures`, `graph`, `search`, locale packs, validation reports) has no defined consumer yet:

- `internal/components/maker` and `internal/components/specialist` are boundary stubs only (`Descriptor()` and nothing else) — there is no running code to derive a real field-level contract from;
- MkDocs + Astro (the actual Library/PDF publication path) consumes canonical `docs/` Markdown and the semantic index directly today, not the bundle — `internal/components/library` was retired by [ADR-008](#) for being an uncalled parallel path, not a bundle consumer to design a contract for;
- no external/third-party consumer exists yet.

Both the review's proposal and design notes are explicit: expanding the bundle before consumers are defined risks producing "attractive but unused generated artifacts," and non-goals explicitly forbid treating the Maker/Specialist stubs as if they were implemented products.

Decision

Bundle expansion is **contract-first**: no field is added to `Bundle` / `BundleUnit` (`internal/components/bundles/bundles.go`) for a given consumer until that consumer has a written contract stating:

1. which bundle fields it reads and why (not "might use");
2. the read pattern (one-shot load, incremental query, streaming);

3. a compatibility expectation (can it tolerate additive-only changes, or does it need a schema version bump on any change);
4. a test that exercises the consumer against a bundle fixture, per `tasks.md`'s "add tests for any new CLI or bundle contract before implementation."

This repo does not yet have a contract meeting that bar for any consumer. Concretely, that means:

- Maker and Specialist stay boundary stubs until their first real use case exists; writing a bundle contract for a stub product would be speculation, not a contract.
- Library's future bundle consumption (if any) needs its own contract before `bundles` grows fields for it — Library reading `docs/` directly today is not itself a reason to add unrelated fields.
- External/third-party consumers need the same bar; `bundleSchema = "rgb-system-bundle/0.1"` stays additive-compatible (new optional fields only) until a real external consumer forces a versioning decision.

Consequences

Positive:

- prevents the bundle from accumulating unused speculative structure;
- keeps `bundles` importing only `internal/components` (the shared leaf contract), per the existing `TestComponentsDoNotImportSiblings` boundary — a contract-first bundle has no reason to reach into a sibling component to guess at its needs.

Negative / accepted costs:

- Maker/Specialist integration stays blocked until product work defines what they actually need, which this ADR does not do — it is a process gate, not a schedule.

Update (2026-08-27)

This ADR's Context section evaluated `internal/components/bundles` (the Go component) but missed a pre-existing, parallel artifact: `docs/core/semantic/projection-manifest.v0.1.json` already declared a `projection.bundle.core-rules.v0_1` projection (added 2026-07-26, a week before this ADR), claiming `"owner": "bundles"` and producing `generated/bundles/core-v2-rules.bundle.json` with a shape already larger than the minimal one described above — the same "attractive but unused generated artifact" risk this ADR warns against, just via a different code path. It had no real consumer (confirmed: no non-test Go code read it) and was never promoted past `"status": "proposed"`. The entry has been removed from the projection manifest; this Decision's contract-first policy now applies without exception across both the `bundles` component and any future `owner: bundles` projection-manifest entries.

Non-negotiable: component import boundaries

Independent of bundle scope, `docs/architecture/components.md`'s enforced rule set (`tests/architecture/architecture_test.go`: `TestComponentsDoNotImportOuterLayers`, `TestComponentsDoNotImportSiblings`, `TestSharedContractStaysLeaf`, `TestEntrypointsGoThroughApp`) stays exactly as-is. No bundle contract, CLI change, or integration work may introduce a sibling-to-sibling import (e.g. `maker` importing `bundles` directly) to shortcut this policy — it must go through `internal/app`, unchanged from ADR-004's original boundary decision.

ADR-008: Retire The Go HTML/Print/PDF Publication Pipeline

Status

Accepted.

Context

The base project Library review found that RGB Library publication actually happens through MkDocs (Library HTML) → Astro (static hosting) → generated PDF downloads, and flagged that `internal/components/library` still described an older, separate publication concept that could mislead future work into treating it as the active path.

Investigating the fate of `internal/components/library` surfaced a wider finding: `internal/components/compiler` (a hand-written ~1,150-line Markdown parser and HTML/print renderer, with its own tests) and the `cmd/rgb-compiler` binary formed the same orphaned pipeline. A repository-wide search found zero consumers of either package's output — no CI workflow, Makefile target required by `check/review-structure/check-fast`, MkDocs config, or Astro/TypeScript file referenced `generated/library/html/**`, `generated/library/print/**`, `generated/library/PDF_EXPORT.md`, or any `compiler/library` package function. `make compile` and `make compile-full` were documented in `README.md` but not wired into any gate. `cmd/rgb-compiler` (unlike `cmd/rgb-tooling`, see [ADR-006](#)) had no real caller either — no test shelled out to it.

This is a different situation from ADR-006's `cmd/rgb-tooling` / `cmd/rgb-compiler` deprecation: there, real callers existed (`tests/semantic_docs/semantic_docs_test.go`, `docs/core/semantic/README.md`) so the binaries were kept working and only marked deprecated. Here, no real caller existed for the compile path at all — keeping it working would have preserved dead weight, not backward compatibility.

Decision

Retire the Go HTML/print/PDF pipeline outright, not deprecate it:

- Delete `internal/components/library` (`library.go`, `site.go`, `pdf_export.go`, and their tests).
- Delete `internal/components/compiler` (Markdown parser, `block.go`, `render_html.go`, `render_print.go`, `render_tree.go`, and their tests).
- Delete `internal/app/compile.go` (`app.Compile`).
- Remove the `compile` subcommand from `cmd/rgb` (`cmd/rgb/main.go`); `rgb compile ...` now returns `unknown subcommand "compile"`.
- Delete `cmd/rgb-compiler` entirely (no ADR-006-style deprecation — it had no real caller to protect).
- Remove `compile` / `compile-full` from `Makefile` and their `README.md` documentation.
- Drop `library` / `compiler` from `internal/app.Components()` and the architecture diagrams (`docs/architecture/components.md`, `docs/architecture/containers.md`).

The documented publication path at the time of this decision was the interpreter-backed Library/PDF path plus Astro. ADR-010 later replaced that active path with `cmd/rgb docs ...` plus Astro, without reviving the deleted packages or the removed `rgb compile` command.

Consequences

Positive:

- removes a substantial parallel implementation (custom Markdown parser + two renderers + site/PDF-instruction writer) that had zero real consumers, eliminating the exact drift risk the Library review warned about ("two publication narratives that should be reconciled");
- `internal/app.Components()` and the architecture diagrams now describe only components with an active call path from `cmd/*`.

Negative / accepted costs:

- this is a real deletion of working, tested code (not a stub) — reversible only through git history, not through a runtime flag or config toggle;
- if a future need for programmatic (non-MkDocs) HTML/print rendering emerges, it will be rebuilt rather than resumed from this code.

Relationship To ADR-006

ADR-006 originally described `cmd/rgb`'s `compile` subcommand and `cmd/rgb-compiler`'s deprecation-not-removal alongside `rgb-tooling`. This ADR supersedes that part of ADR-006: `compile` and `cmd/rgb-compiler` are removed, not deprecated. ADR-006's decision about `cmd/rgb-tooling` (kept deprecated, real callers) is unaffected and still stands.

ADR-009: Retain The MkDocs/Python Publication Exception Until Replacement Parity

Status

Superseded by [ADR-010](#).

Historical status before supersession: Accepted.

Context

ADR-005 established the project's zero external dependency posture and then added a narrow documentation-PDF exception on 2026-07-31. That exception allows Python documentation-build dependencies required by the MkDocs PDF plugin, operating-system rendering libraries documented for that plugin, and generated PDFs under the landing download surface.

The project-owned release validation path has since been moved to Go: release manifest generation, checksum verification, PDF metadata checks, TOC checks, and raster smoke checks no longer depend on Python or Pillow. That leaves the renderer/publication stack itself as the remaining Python surface:

- `make docs-build` uses `python3 -m mkdocs` for the HTML Library;
- `make docs-pdf` uses MkDocs plus `mkdocs-to-pdf` for bilingual PDF downloads;
- CI uses `actions/setup-python` for documentation and deployment jobs;
- `docs-build/requirements-docs.txt` and `docs-build/requirements-docs-pdf.txt` define the Python dependency profiles;
- Dependabot tracks the `/docs-build` `pip` ecosystem while the exception remains active.

ADR-008 is also binding context. It retired the previous Go HTML/print/PDF pipeline because it had no active caller and would have preserved a parallel, orphaned publication narrative. A future non-Python renderer must therefore be a new justified architecture decision with real consumers and parity evidence, not a silent resurrection of `internal/components/compiler`, `internal/components/library`, `cmd/rgb-compiler`, or `rgb compile`.

Decision

Keep the MkDocs/Python publication exception for now.

The current publication topology remains:

```
docs/**
|
+-- MkDocs strict build -----> web/landing/public/library/**
|
+-- MkDocs + mkdocs-to-pdf + Poppler ---> web/landing/public/downloads/*.pdf
                                     |
                                     v
                                Go-owned release artifact gates
```

Retiring Python from documentation publication requires a later ADR or an explicit amendment to this ADR. That later decision must identify the replacement topology and provide parity evidence before deleting the current Python surfaces.

The replacement must preserve or deliberately supersede all current public release behavior:

- HTML Library generation;
- Astro landing packaging of the Library;
- English and PT-br PDF generation;
- stable `latest` PDF aliases;
- versioned PDF files;
- release manifest generation;
- aggregate `SHA256SUMS` and per-version `.sha256` files;
- PDF editorial smoke checks.

Until that parity exists, do not remove or weaken:

- `PYTHON`, `MKDOCS`, `docs-install`, `docs-pdf-install`, `docs-build`, or `docs-pdf` in the Makefile;
- `docs-build/requirements-docs.txt`;
- `docs-build/requirements-docs-pdf.txt`;
- Python setup and pip caching in CI documentation/deployment jobs;
- the Dependabot `pip` entry for `/docs-build`.

Consequences

Positive:

- keeps the current public documentation and PDF release path stable;
- avoids deleting a working renderer without a proven replacement;
- keeps ADR-005's exception narrow and explicit;
- prevents accidental reversal of ADR-008 by forbidding a silent revival of the deleted Go compiler/library pipeline;

- leaves release artifact validation Go-owned and independent from the Python renderer exception.

Negative / accepted costs:

- the repository still needs Python for documentation publication;
- `docs-build/` remains a dependency surface that CI and Dependabot must track;
- fully zero-Python publication remains deferred until a later ADR selects and proves a replacement topology.

Replacement Requirements

Any future proposal to retire MkDocs/Python must include at least:

1. A renderer topology that does not reintroduce the ADR-008 deleted pipeline by implication.
2. Local validation equivalent to or stronger than:
3. `make docs-build`;
4. `make landing-build`;
5. `make docs-pdf`;
6. `make release-artifact-manifest`;
7. `make release-artifact-check`;
8. `make pdf-editorial-check`.
9. CI changes proving the replacement runs without `actions/setup-python`, pip cache, or `docs-build/requirements-*.txt`.
10. A deletion plan for obsolete Python publication files only after parity passes.
11. Updated public documentation describing the new publication path.

Full PDF/UA tagging, durable visual-regression baselines, and deeper accessibility checks remain future quality work. They are desirable, but they are not prerequisites for keeping the current MkDocs/Python exception active.

ADR-010: Go-Owned Publication Without Python

Status

Accepted.

Context

ADR-005 established the project's zero external dependency posture and later allowed a narrow documentation publication exception for the former renderer stack. ADR-008 removed the orphaned Go HTML/print/PDF compiler path. ADR-009 then kept the former exception only until a replacement topology proved parity.

The remaining interpreter-backed surface was publication:

- documentation HTML rendering;
- PDF download publication;
- CI setup for the renderer;
- dependency update tracking for that renderer.

Release metadata and editorial validation were already Go-owned through `cmd/rgb release manifest`, `cmd/rgb release check`, and the tooling package.

Decision

Retire the interpreter-backed publication exception and make publication Go-owned.

The active topology is now:

```
docs/core/**
|
+-- cmd/rgb docs library -----> web/landing/public/library/**
|
+-- cmd/rgb docs pdf -----> web/landing/public/downloads/**
                             |
                             v
                        cmd/rgb release manifest/check
```

Astro remains the static landing package and hosting surface. It packages the generated public Library and PDF downloads from `web/landing/public/**`.

This is a new publication boundary, not a revival of the ADR-008 deleted compiler/library packages or the removed `rgb compile` command.

Consequences

Positive:

- project publication no longer installs or runs the retired interpreter stack;
- CI and local Make targets use Go plus the existing Astro/Node landing workspace;
- dependency update tracking no longer needs the retired renderer ecosystem;
- the public Library is generated through the unified `cmd/rgb` CLI topology;
- release manifest and checksum behavior remains Go-owned.

Accepted costs:

- the Go HTML renderer intentionally supports a constrained Markdown subset based on the current public `docs/core/**` files;
- PDF publication is a Go-owned publishing step for reviewed PDF assets, not a full general-purpose PDF layout engine;
- Poppler remains an external command-line validation dependency for PDF editorial smoke checks. It is not part of publication rendering and does not reintroduce the retired interpreter runtime.

Replacement Parity

The replacement preserves:

- HTML Library generation from canonical `docs/core/**`;
- Astro packaging of the Library;
- English and PT-br public documentation surfaces;
- stable `latest` PDF aliases;
- versioned PDF files;
- release manifest generation;
- aggregate `SHA256SUMS` and per-version `.sha256` files;
- PDF editorial smoke checks.

Supersession

This ADR supersedes ADR-009 for active publication behavior. ADR-009 remains as the historical record of why the exception was retained until this replacement existed.

ADR-011: Split License Model

Status

Accepted.

Context

The repository contains two materially different kinds of work:

- executable software surfaces such as Go packages, shell scripts, CI, and Astro application code;
- creative and rules-authority surfaces such as RPG rules, examples, documentation, diagrams, public copy, generated Library/PDF content, and art.

The prior single CC BY 4.0 repository license was suitable for creative rules text, but it was less conventional for software reuse and downstream tooling.

Decision

Adopt a split model:

SURFACE	LICENSE
Source code, scripts, CI, and tooling	MIT or Apache-2.0
Rules text, examples, documentation, diagrams, web copy, generated content, and art	CC BY 4.0

`LICENSES.md` is the repository surface map. The root `LICENSE` file points to the concrete license texts.

Consequences

- Software consumers can use the code under a conventional permissive software license.
- Rules and documentation retain attribution requirements appropriate for creative RPG content.
- New files should be classified by surface ownership before publication.
- Ambiguous generated artifacts inherit the license of their source surface.

ADR-012: MIT-Only License

Status

Accepted.

Context

ADR-011 adopted a split licensing model: source code, scripts, CI, and tooling were dual-licensed under MIT or Apache-2.0, at the user's option, while rules text, examples, documentation, diagrams, web copy, and generated Library/PDF content were licensed under Creative Commons Attribution 4.0 International (CC BY 4.0), which requires attribution.

The goal has since narrowed: a single, maximally permissive license across the entire repository, with unrestricted open-source use, and with the copyright notice itself naming the actual author rather than only the project name.

Decision

Adopt MIT as the sole license for the entire repository — code, rules text, documentation, diagrams, web copy, generated content, and art alike. The Apache-2.0 option and the CC BY 4.0 license are retired from active use.

The root `LICENSE` file holds the full MIT license text directly (not an index to separate license files), with:

```
Copyright (c) 2026 Sergio Lacerda (RGB System)
```

`LICENSE-MIT`, `LICENSE-APACHE-2.0`, `LICENSE-CC-BY-4.0`, and `LICENSES.md` are removed — there is no split model left to map or duplicate.

Consequences

- The repository's licensing surface collapses to a single `LICENSE` file.
- Anyone may use, copy, modify, merge, publish, distribute, sublicense, and sell all repository content, subject only to MIT's copyright-notice retention condition — matching the request for unrestricted open-source access.

- The CC BY 4.0 attribution *requirement* for rules text, examples, docs, diagrams, and art is dropped. This is an explicit, accepted trade-off in favor of being fully unrestricted, not an oversight.
- The Apache-2.0 explicit patent grant is no longer offered for code.
- All prose references to the split model (`README.md` , `docs/README-details.md`) are updated to point at `LICENSE` directly.

Supersession

This ADR supersedes ADR-011. ADR-011 remains as the historical record of why the split model was originally adopted.

ADR-013: Skill Distribution Via The Existing Publication Pipeline

Status

Accepted.

Context

A proposal to distribute the RGB Specialist skill as a downloadable `.zip` (captured in an internal pending-analysis note) suggested a git-tag-triggered GitHub Actions job publishing the archive as a GitHub Release asset, with a dedicated install page and shell commands to unzip it into an `~/.agents/skills`-style location.

This repository has no git-tag-triggered release job and no `action-gh-release`-style step anywhere in `.github/workflows/ci.yml`. Publication instead happens on every push to `main`: a `deploy` job bundles generated documentation artifacts into a single static site (`web/landing/dist`) and publishes it to GitHub Pages via `actions/upload-pages-artifact` + `actions/deploy-pages`.

ADR-010 already established that topology for the two existing publication outputs:

```
docs/core/**
|
+-- cmd/rgb docs library -----> web/landing/public/library/**
|
+-- cmd/rgb docs pdf -----> web/landing/public/downloads/**
                             |
                             v
                        cmd/rgb release manifest/check
```

ADR-008 separately established that this project retires orphaned, parallel publication pipelines rather than let them accumulate alongside the active one. Adding a GitHub-Release-based flow for skill distribution would reintroduce exactly that pattern: a second publication mechanism, with its own trigger, permissions, and versioning scheme, running alongside the Pages-based one that already serves the Library and PDFs.

Separately, `cmd/rgb` is this project's unified CLI (ADR-006), and it is never invoked by end users directly — only by `make` targets and CI jobs. Any skill distribution mechanism needs to preserve that: no new user-facing installer CLI for this project.

`internal/components/tooling/release_artifacts.go`'s `WriteReleaseArtifactManifest`/`CheckReleaseArtifacts` — the existing manifest/checksum machinery for PDF downloads — hardcodes an expected file list of exactly four PDF files (two locales, `latest` + versioned) and runs PDF-specific editorial validation (`pdfinfo`/`pdftotext`/`pdftohtml`/`pdftoppm`, page-count, TOC-link, and raster-luminance checks). It is not a generic "checksum any artifact set" utility and cannot be pointed at a skill `.zip` without change.

Decision

Skill distribution joins the existing Go-owned publication topology (ADR-010) as a third artifact type, published the same way the Library and PDFs already are — not via GitHub Releases:

```
skills/<skill>/**
|
+-- cmd/rgb docs skill -----> web/landing/public/downloads/**
                                |
                                v
                        a lightweight skill-artifact manifest/checksum step
                        (separate from, not a reuse of, the PDF-specific
                        release manifest/check machinery above)
```

- A skill package (e.g. `skills/specialist/`) is zipped by a new `cmd/rgb docs skill` sub-command, with the archive's top-level entry being the skill's own directory name — never a repo-root-relative or auto-generated wrapper folder.
- The `.zip` (`<name>-<version>.zip`, `<name>-latest.zip`) is written to `web/landing/public/downloads/`, the same directory the PDFs already use, and is published as a static file through the existing push-to-`main` → GitHub Pages `deploy` job. No new CI trigger, secret, or permission is introduced.
- Manifest/checksum generation for the skill `.zip` is a new, minimal step (SHA256 + byte count only), deliberately kept separate from `WriteReleaseArtifactManifest`/`CheckReleaseArtifacts` rather than generalizing those functions — the PDF checker's entire design center is PDF-specific editorial validation that does not apply to a skill archive, and coupling the two risks regressing PDF checks when the skill path changes.
- No new CLI is introduced for end users. `cmd/rgb docs skill` is invoked the same way `cmd/rgb docs library`/`cmd/rgb docs pdf` already are: through a Makefile target, from CI, never directly by a person installing the skill.

Consequences

Positive:

- skill distribution reuses an already-reviewed publication boundary instead of introducing a second one;

- no new GitHub Actions triggers, permissions, or secrets;
- naming/versioning for the skill archive mirrors the existing PDF convention (`<name>-<version>`, `<name>-latest`), keeping the public `downloads/` directory self-consistent;
- the PDF-specific release-manifest/check machinery stays focused on PDF editorial validation and is not stretched to cover an unrelated artifact shape.

Accepted costs:

- a new, separate manifest/checksum code path is needed for the skill `.zip` rather than a single shared implementation across all downloadable artifacts;
- distribution here covers packaging/publication only — skill package *content* (what `skills/specialist/` actually contains, and whether it has a runnable contract) is defined by separate, independent work and is out of scope for this decision;
- multi-runtime skill packaging (for example a Codex- or Gemini-specific layout) is explicitly out of scope; this decision targets the Claude Code skill-loading convention already used by `skills/specialist/` and `skills/maker/` in this repository.

Relationship To Prior ADRs

This ADR extends ADR-010's topology rather than amending it: ADR-010's Library/PDF flow is unchanged, and this decision adds a third, independent branch alongside it. It reaffirms ADR-008's rejection of parallel/orphaned publication pipelines and ADR-006's unified-CLI boundary (`cmd/rgb` is never an end-user-facing installer).

ADR-014: RGB Specialist Bundled References Snapshot

Status

Accepted.

Context

`skills/specialist/` grounds its answers in `generated/ai-context/core-specialist-pack.json` and `docs/core/semantic/**` (see ADR-002). Both are repo-relative: if the `skills/specialist/` folder is copied outside this monorepo — which ADR-013 now makes possible by publishing it as a downloadable `.zip` — it loses access to the canonical rule documentation it depends on to answer questions and cite a source.

The user asked for the package to be self-contained, carrying canonical documentation material with it, rather than depending on sibling paths in this monorepo.

Decision

`skills/specialist/references/rgb-system.md` is added as a bundled copy of the four canonical rule documents (`docs/core/en/**` and `docs/core/PT-br/**`), consolidated verbatim with each section labeled by its original source path.

This is a **hand-maintained snapshot**, not a generated artifact:

- `docs/core/**` remains the single source of truth (ADR-002) — the bundled copy never overrides it;
- `SKILL.md`'s grounding-source precedence uses the bundled copy only as a fallback (inside the monorepo) or as the sole available source (running standalone, outside the monorepo) — citation is still required in every case;
- no automated regeneration step exists yet for this file (unlike `generated/ai-context/*`, which is regenerated via `make generate`). Adding one is a config/tooling change, deferred to a separate

task (Task 6 of the parallel skill-materialization mission) — it touches `config.yaml`, which is out of this mission's documentation-only scope.

Consequences

Positive:

- the skill package answers questions and cites a source even when copied outside this monorepo, satisfying the self-contained requirement without changing the existing consumer contract or procedures;
- the precedence order keeps the higher-trust, ID-traceable sources (compiled pack, semantic index) as first choice whenever they are available, so nothing regresses for in-monorepo use.

Accepted costs:

- `references/rgb-system.md` and `docs/core/**` can drift until the automation from Task 6 exists — anyone updating `docs/core/**` should know the bundled copy needs a manual refresh until then;
- this is intentionally scoped to content only. It does not decide how the folder gets packaged or published; that is ADR-013's decision.

Update (2026-08-27)

Task 6 is closed: `skills/specialist/references/rgb-system.md` is now regenerated by `make generate` (`internal/components/tooling.GenerateSkillSnapshot`, reading the input list from `skills/specialist/config.yaml`'s `sources.bundled_snapshot_inputs`) and drift-checked in CI (`scripts/ci/check-generated-drift.sh`), which `deploy` already depends on. The "accepted cost" above — drift between this file and `docs/core/**` until Task 6 landed — no longer applies: the file is a generated artifact now, not hand-maintained, and a stale snapshot blocks deployment instead of publishing silently.

Relationship To ADR-013

ADR-013 decided how `skills/specialist/` gets zipped and published (reusing the existing Pages-based publication pipeline). This ADR decides what the folder needs to contain to still be useful once distributed that way — a self-contained copy of the canonical rules it grounds answers in. The two are independent and do not amend each other: ADR-013 governs packaging/publication, this ADR governs package content.

ADR-015: Skill Runtime Split — LLM Instruction Layer vs. Go Component Layer

Status

Accepted.

Context

`skills/specialist/**` and `skills/maker/**` both ship a complete skill contract (`SKILL.md`, procedures/schemas, templates, config) but neither has a working runtime. `internal/components/specialist/specialist.go` and `internal/components/maker/maker.go` are both 13-line stubs exposing only a `Descriptor()` — there is no behavior on either side of the boundary today.

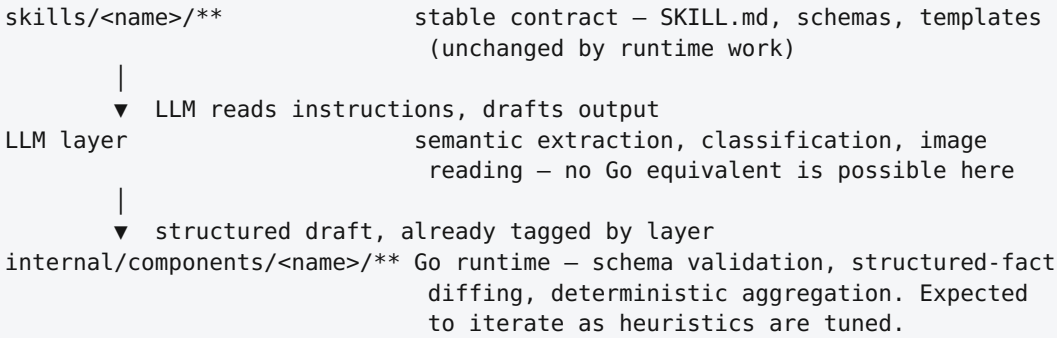
The accepted Specialist-first skill roadmap gated Maker's runtime work behind three conditions: (a) Specialist has a source-trace benchmark, (b) the bundle/search context shape is stable, (c) Maker's provenance/canon schemas are accepted. A 2026-08-27 runtime-base review re-checked these against the current repository:

- (b) and (c) are met — `generated/ai-context/core-specialist-pack.json` / `docs/core/semantic/core-v2.index.json` are in stable use, and Maker's four schemas (`entity-package`, `maker-report`, `visual-observation`, `conflict-report`) exist and are internally consistent.
- (a) is met only **structurally** — `skills/specialist/benchmark/golden-qa.yaml` plus `tests/fixtures/specialist_golden_qa_test.go` validate that the benchmark dataset cites real semantic IDs, but no Specialist runtime exists to exercise the benchmark behaviorally, because Specialist has no runtime either.

That review also found that "use Specialist as the base" only holds for the contract layer (`SKILL.md`/schemas/procedures shape) — there is no proven runtime pattern in this repo to copy, for either skill.

Decision

Split any skill's runtime, once it needs one, into two layers:



Applied to Maker specifically, per package kind:

KIND	LLM LAYER	GO LAYER (INTERNAL/COMPONENTS/MAKER/**)
Entity Package	extract facts/inferences from notes	validate against entity-package.schema.yaml
Maker Report	thin prose rollup	aggregate counts over already-classified packages
Visual Observati-on	read appearance from an image	enforce appearance-only field scope (image never claims a fact fi-eld)
Conflict Report	flag disagreement during extracti-on	diff structured fields across packages sharing an entity ID

The contract layer (skills/maker/**) is the stable surface and is not expected to change as part of this work. The Go component layer (internal/components/maker/**) is where iteration is expected — differ heuristics, aggregation buckets, and validation strictness will be tuned as real usage surfaces edge cases.

Gate status: conditions (b) and (c) of the Maker Deferral Criteria are treated as met; condition (a) is treated as met for starting Maker's Go-side runtime work (validators, differ, aggregator — none of which require a working Specialist runtime to build or test), but not as evidence that Maker's own LLM-side behavior has been benchmarked.

Consequences

Positive:

- Isolates expected instability (differ/aggregator/validator heuristics) to one package boundary, so iterating on it does not put the already-accepted skills/maker/** contract at risk of drifting out of sync with runtime behavior.
- Establishes a pattern reusable by Specialist (and any future skill) once it grows a real runtime, rather than a Maker-specific one-off.
- Continues the split already used for Specialist's existing tooling (bundle generation and the golden-qa structural check are Go; explaining a rule is LLM) instead of introducing an unrelated third pattern.

Negative / accepted costs:

- No implementation of this split exists yet for either skill — this ADR sets policy ahead of a first concrete build.
- Building `internal/components/maker/**`'s validators, differ, and aggregator, and wiring Maker's LLM-side procedures into an invocable runtime, remain open implementation work — out of scope for this ADR and for the Strategist mission that produced it (implementation_handoff, not documentation).

Non-Goals

This ADR does not authorize or perform any Go source change under `internal/components/maker/**`, nor any change to `skills/maker/**`'s existing contract. It records the architectural decision only.

ADR-016: Reinstate A Narrow External PDF Authoring Exception

Status

Accepted.

Context

A 2026-08-27 PDF editorial-quality review re-examined a pending critique that proposed adding a headless-Chromium/Playwright HTML-to-PDF pipeline to achieve "editorial quality" (cover, table of contents, RGB-vector rule callout boxes) for the RGB Library PDF.

Two things emerged from direct verification, not just from reading the critique:

1. **Most of the claimed gap does not exist.** Inspecting the currently shipped PDF (`web/landing/public/downloads/rgb-system-core-v2-*.pdf`) directly — via `pdftotext`, `pdf-tohtml -xml`, and a rasterized page render — shows it already has a correctly paginated table of contents, 146 working internal hyperlinks on pages 1–3 alone, and running headers/footers (`page/total`). Only two things are actually missing: a designed cover (no RGB vector glyph, no version/license/date metadata — plain text only) and colored R/G/B rule callout boxes.
2. **The proposed fix conflicted with existing decisions.** The headless-Chromium pipeline directly conflicts with [ADR-005](#) (which already named and rejected headless-Chromium PDF generation once, in mission `docs-compiler-html-pdf-scope-20260726`) and [ADR-010](#) (which retired the interpreter-backed publication path and states PDF publication is "a Go-owned publishing step for reviewed PDF assets, not a full general-purpose PDF layout engine").

Deeper verification found the real problem underneath: the shipped PDF's own metadata reports `Producer: WeasyPrint 62.3`, and `git log` traces it to a pre-ADR-010 commit (`c2d2a5e`, "implement automated PDF generation workflow with MkDocs"). A repository-wide search finds zero remaining trace of that toolchain — no `mkdocs.yml`, no `requirements-docs*`, no `weasyprint` or `mkdocs-pdf` reference anywhere. The tool that produced today's good-quality PDF body was fully deleted when ADR-010 retired the automated publication path, without a documented replacement for *authoring* PDF content (as opposed to *publishing* it, which ADR-010's Go-owned pipeline still does correctly). Nobody today has a reproducible way to produce the next PDF release at all, cover/callouts aside.

ADR-005's original 2026-07-31 addendum already permitted exactly this class of exception (Python documentation-build dependencies, OS rendering libraries, a dedicated `docs-pdf`/`pdf-build` Make target, generated PDFs under the landing download surface) before ADR-010 retired it on the premise that the Go-owned replacement had reached parity. That parity claim holds for copy/validate/manifest/checksums, but not for authoring the PDF body's editorial content — nothing replaced MkDocs+WeasyPrint's role there.

Decision

Reinstate a **narrow, external, non-Go, non-CI-blocking** PDF authoring exception:

- A PDF authoring tool of this class (MkDocs + WeasyPrint, or an equivalent the maintainer chooses) may run **outside** the Go module graph and outside the default `make`/CI build, as a dedicated, manually-invoked release step — producing one reviewed PDF per release, per language.
- That authoring step's template must add what's actually missing: a designed cover (RGB vector glyph, version, language, license, release date) and CSS-based callout/admonition styling for R/G/B rule boxes. It does not need to reproduce anything else — the existing TOC pagination, internal links, and running headers/footers already work and must be preserved, not rebuilt.
- The resulting reviewed PDF is supplied to the existing, unchanged Go pipeline via `cmd/rgb docs pdf --source-en / --source-pt-br` (`make docs-pdf`), exactly as today. `internal/components/publication/pdf.go`'s copy-and-validate behavior does not change.
- `internal/components/tooling/release_artifacts.go`'s Poppler-based editorial smoke checks (`pdftotext`/`pdftohtml`/`pdftoppm`) remain the Go-owned validation gate over whatever the external tool produces.

This does **not** allow:

- any new Go module dependency (`go.sum` entry) for PDF generation or manipulation;
- headless-browser automation as part of the Go/CI build;
- treating the external authoring tool's output as canonical documentation — canonical source remains `docs/core/**` Markdown, unchanged (ADR-001);
- making any default build target (`make build`, `make test`, `make release-check`)'s non-PDF gates) depend on the external tool's presence.

Consequences

Positive:

- Closes a real reproducibility gap: the next PDF release has a documented path again, instead of depending on tooling that was silently deleted.
- Scopes the actual work correctly: template/cover/callout design, not a new pipeline or a new class of Go/CI dependency.
- Keeps ADR-010's substance intact — Go still owns copy, validate, manifest, and checksums; only PDF *authoring* (never publication) sits outside Go.
- Prevents re-litigating the headless-Chromium-vs-ADR-005 conflict from scratch the next time PDF editorial quality comes up.

Negative / accepted costs:

- Reintroduces an external, non-Go toolchain dependency for PDF authoring — the same tradeoff ADR-005's original addendum already accepted, now scoped more narrowly (cover + callout templates only, not full pipeline automation).
- Setting up the authoring tool, and designing the cover/callout templates, remain open implementation/authoring work — out of scope for this ADR and for the Strategist mission that produced it.

Non-Goals

This ADR does not authorize any Go module dependency, any change to `internal/components/publication/**` or `internal/components/tooling/**`, or any change to the canonical Markdown authority model (ADR-001). It records the authoring-exception decision only; the toolchain setup and the actual next PDF release are separate, unauthorized-by-this-ADR implementation work.

Supersession

This ADR narrows and partially reinstates the addendum ADR-010 retired — scoped specifically to PDF *authoring* (cover/callout templates), not to the full automated interpreter-backed publication pipeline ADR-010 removed. ADR-010's Go-owned publication decision remains otherwise unchanged.

Architecture Decision Records

This directory contains project-wide architecture decisions for RGB System.

- [ADR-001: AI-First Documentation Authority](#)
- [ADR-002: RGB Core V2 Design Control](#)
- [ADR-003: Documentation L10n Maintenance Pattern](#)
- [ADR-004: Modular Monolith In A Single Monorepo](#)
- [ADR-005: Zero External Dependency Posture](#)
- [ADR-006: `cmd/rgb` As The Unified CLI](#)
- [ADR-007: Bundle Expansion Requires A Consumer Contract First](#)
- [ADR-008: Retire The Go HTML/Print/PDF Publication Pipeline](#)
- [ADR-009: Retain The MkDocs/Python Publication Exception Until Replacement Parity](#)
- [ADR-010: Go-Owned Publication Without Python](#)
- [ADR-011: Split License Model](#)
- [ADR-012: MIT-Only License](#)
- [ADR-013: Skill Distribution Via The Existing Publication Pipeline](#)
- [ADR-015: Skill Runtime Split — LLM Instruction Layer vs. Go Component Layer](#)
- [ADR-016: Reinstate A Narrow External PDF Authoring Exception](#)

Explosivos

Explosivos representam **armas de área** usadas para causar dano ou efeitos táticos no Sistema RGB.

Granadas e dispositivos explosivos podem causar dano direto, efeitos de controle ou alterar o campo de batalha.

Diferente de armas convencionais, explosivos geralmente **afetam múltiplos alvos em uma área**.

Tipos de Detonação

Granadas podem detonar em diferentes tempos de ativação.

Tipo de Detonação	Tempo	
-----	-----Imediata	mesma rodada
Curta	1 rodada	
Atrasada	até 2 rodadas	

O tempo de detonação pode ser escolhido pelo personagem ao arremessar a granada, dependendo do equipamento utilizado.

Tipos de Granadas

O Sistema RGB permite diferentes tipos de explosivos com efeitos distintos.

Tipo	Efeito	
-----	-----Explosiva	dano 8 em área
Elétrica	dano 2 + possível interferência	
Sonora	1d5 rodadas atordoado	
Imobilizante	1d5 rodadas imobilizado	

Granada Explosiva

- causa dano físico em área
- dano pode ser reduzido por **armadura** ou **cobertura**

Granada Elétrica

- causa dano baixo
- pode afetar equipamentos eletrônicos ou sistemas energéticos

Granada Sonora

- não causa dano significativo
- provoca **atordoamento temporário**

Granada Imobilizante

- impede movimento temporariamente
- usada para **controle de área**

Interação com o Sistema RGB

Explosivos interagem com o sistema de combate RGB da seguinte forma:

R → pode aumentar o alcance ou precisão do arremesso
G → influencia posicionamento e evasão da área
B → pode absorver parte do dano através de escudos

O Mestre pode ajustar esses efeitos dependendo do cenário.

Interação com o Modelo de Dano

Quando uma granada causa dano direto, o fluxo de resolução segue o modelo padrão do RGB:

Dano da Granada
↓
Penetração (se aplicável)
↓
Redução por Armadura
↓
Absorção por Escudo
↓
Dano Restante → Personagem

Uso Tático

Explosivos são especialmente úteis para:

- controle de área
- dispersar grupos de inimigos
- remover inimigos de cobertura

- interromper posições defensivas

Por afetarem áreas maiores, explosivos costumam **alterar rapidamente a dinâmica do combate**.

Filosofia de Design

Explosivos no sistema RGB seguem três princípios:

- **impacto tático alto** – alteram rapidamente o campo de batalha
- **regras simples** – poucos valores para fácil resolução
- **modularidade** – novos tipos podem ser adicionados conforme o cenário

Veja Também

- [Armas de Fogo](#)
- [Modelo de Dano](#)
- [Ataque e Defesa](#)

← [Voltar para README](#)

Armas de Fogo

Armas de fogo representam as **principais armas de combate à distância** em cenários modernos e táticos do sistema RGB.

Diferente de armas corpo a corpo, armas de fogo **não adicionam o vetor R (Vermelho) do usuário ao dano**. O dano é determinado pela própria arma, representando o poder balístico e a munição utilizada.

Armas de fogo interagem com o **Sistema de Combate RGB** e seguem o **Modelo de Dano** padrão do sistema.

Estatísticas da Arma

Cada arma de fogo é definida por quatro atributos principais:

Impacto	→ impacto base causado pela arma
Capacidade	→ número de munições no carregador
Recarga	→ ações necessárias para recarregar
Tipo	→ categoria da arma

Tabela de Armas de Fogo

Tipo	Impacto	Capacidade	Recarga
Pistola Compacta	4	12	ação menor
Pistola	4	15	ação menor
Revólver	5	6	ação
SMG	5	30	ação
Rifle de Assalto	7	30	ação
Rifle Pesado / DMR	8	20	ação
Rifle de Precisão	9	10	ação
Metralhadora	7	100	2 ações

Papéis das Armas

Diferentes armas cumprem papéis táticos distintos.

Pistolas

- armas leves e secundárias
- recarga rápida
- eficientes em curta distância

SMGs

- alta cadência de tiro
- eficazes em curta e média distância

Rifles de Assalto

- equilíbrio entre dano e capacidade
- eficazes na maioria das situações de combate

Rifles Pesados / Precisão

- dano elevado
- menor capacidade de munição
- utilizados para funções especializadas

Metralhadoras

- fogo de supressão contínuo
- grande capacidade de munição
- tempo de recarga maior

Interação com o Sistema RGB

Armas de fogo interagem indiretamente com os vetores RGB.

R → melhora capacidade ofensiva ou controle da arma
G → influencia posicionamento e mobilidade em combate
B → fornece proteção defensiva através de escudos

Entretanto, a **Fonte de Impacto** de uma arma de fogo vem da própria arma, e não do valor de R do personagem, salvo regra ou habilidade explícita.

Interação com o Modelo de Dano

Quando um ataque com arma de fogo atinge um alvo, o fluxo de dano do sistema RGB é aplicado:

```
Fonte de Impacto
↓
Penetração (se aplicável)
↓
Redução por Armadura
↓
Absorção por Escudo
↓
Dano Restante → Personagem
```

Isso mantém o combate consistente com o restante do sistema.

Filosofia de Design

Armas de fogo no RGB seguem três princípios:

- **clareza** — estatísticas simples de entender
- **variedade tática** — armas diferentes cumprem funções diferentes
- **compatibilidade com o sistema** — integração direta com o modelo de dano RGB

Isso permite que armas de fogo sejam poderosas sem tornar o sistema complexo.

Veja Também

- [Combate](#)
- [Alcance de Armas](#)
- [Modelo de Dano](#)

← [Voltar para README](#)

Armas Corpo a Corpo

Armas corpo a corpo representam **armas utilizadas em combate direto** no Sistema RGB.

Diferente de armas de fogo, armas corpo a corpo podem **utilizar o vetor R (Vermelho) do personagem como parte da Fonte de Impacto** quando a arma ou procedimento permitir.

Isso representa força física, impacto e habilidade marcial do usuário.

Fonte de Impacto

A Fonte de Impacto de uma arma corpo a corpo normalmente é calculada somando o valor de **R do personagem** ao bônus da arma.

Fonte de Impacto = R + bônus da arma

Exemplo:

R = 3
Espada Longa = +3

Fonte de Impacto = 6

Tabela de Armas Corpo a Corpo

Arma	Bônus de Dano	
-----	-----	Faca +1
Adaga	+1	
Espada Curta	+2	
Espada Longa	+3	
Machado	+3	

Interação com o Sistema RGB

Armas corpo a corpo estão diretamente ligadas ao vetor **R (Vermelho)**.

R → determina a pressão e pode contribuir para a Fonte de Impacto
G → influencia mobilidade, timing e posicionamento
B → preserva continuidade por bloqueios, escudos e estabilização

Personagens com valores altos de **R** tendem a ser mais eficazes em combate corpo a corpo.

Interação com o Modelo de Dano

Quando um ataque corpo a corpo acerta um alvo, o fluxo de dano segue o modelo padrão do RGB.

```
Fonte de Impacto (R + arma)
↓
Penetração (se aplicável)
↓
Redução por Armadura
↓
Absorção por Escudo
↓
Dano Restante → Personagem
```

Uso Tático

Armas corpo a corpo são particularmente eficazes em:

- combate em curta distância
- espaços confinados
- situações onde armas de fogo não podem ser utilizadas

Elas também permitem combate silencioso ou ataques surpresa.

Filosofia de Design

Armas corpo a corpo no RGB seguem três princípios:

- **simplicidade** — cálculo direto baseado no vetor R
- **equilíbrio** — dano proporcional à força do personagem
- **compatibilidade** — integração direta com o modelo de dano RGB

Veja Também

- [Combate](#)
- [Modelo de Dano](#)
- [Armas de Fogo](#)

← [Voltar para README](#)

Penetração

Penetração representa a capacidade de uma arma **superar a proteção de armaduras** no Sistema RGB.

Armas com maior poder balístico ou impacto físico possuem maior capacidade de atravessar proteções.

Cálculo de Penetração

A penetração reduz temporariamente o valor de armadura do alvo durante o cálculo de dano.

$$\text{Armadura Efetiva} = \text{Armadura} - \text{Penetração}$$

Exemplo:

Armadura = 4
Penetração da arma = 2

Armadura efetiva = 2

Se o valor de armadura efetiva chegar a **0 ou menos**, a armadura não reduz o dano.

Tabela de Penetração de Armas

Arma	Penetração
-----	-----
Pistola	1
SMG	2
Carabina	2
Rifle de Assalto	3
Rifle de Precisão	4
Metralhadora	3

Interação com o Modelo de Dano

A penetração é aplicada **antes da redução de dano pela armadura**.

Fluxo completo de resolução de dano:

Fonte de Impacto
↓
Penetração
↓
Redução por Armadura
↓
Absorção por Escudo
↓
Dano Restante → Personagem

Interação com o Sistema RGB

Penetração está relacionada principalmente ao impacto da arma, mas interage indiretamente com os vetores RGB.

R → pode aumentar impacto em armas corpo a corpo quando a Fonte de Impacto permite
G → permite evitar ataques por mobilidade, timing e posicionamento
B → fornece preservação por bloqueios, escudos e estabilização

Armas com maior penetração são mais eficazes contra alvos fortemente protegidos.

Filosofia de Design

O sistema de penetração no RGB segue três princípios:

- **simplicidade** — cálculo direto e fácil de aplicar
- **equilíbrio** — armas diferentes possuem capacidades distintas contra armaduras
- **compatibilidade** — integração direta com o modelo de dano RGB

Isso permite diferenciar armas sem adicionar complexidade excessiva ao combate.

Veja Também

- [Armas de Fogo](#)
- [Modelo de Dano](#)
- [Armadura](#)

← [Voltar para Armas de Fogo](#)

Alcance de Armas

O alcance define a **eficácia de armas à distância em diferentes distâncias de combate** no Sistema RGB.

Essas regras permitem representar perda de precisão e eficiência conforme o alvo se afasta.

Modificadores de Distância

Ataques com armas de fogo recebem modificadores dependendo da distância ao alvo.

Distância	Modificador	
-----	-----Curto	0
Médio	-1 R	
Longo	-2 R	

Esses modificadores representam a dificuldade crescente de acertar alvos distantes.

Tabela de Alcance (metros)

Arma	Curto	Médio	Longo				
-----	-----	-----	--	Pistola Compacta	10	20	35
Pistola	15	30	50				
Revólver	20	40	60				
SMG	20	50	80				
Rifle de Assalto	40	120	250				
Rifle Pesado	60	200	400				
Rifle de Precisão	150	500	1000				
Metralhadora	60	200	400				

Alcance Extremo (Opcional)

Ataques além do alcance longo ainda podem ser tentados, mas são significativamente mais difíceis.

Alcance extremo → -4 R

O Mestre decide se o alvo ainda está visível e se o disparo é plausível.

Interação com o Sistema RGB

O alcance se relaciona principalmente com o vetor **R (Vermelho)**, que representa precisão ofensiva.

R → precisão e eficácia do ataque
G → posicionamento para obter melhor linha de tiro
B → proteção defensiva contra ataques

Personagens com maior valor de **R** tendem a lidar melhor com penalidades de distância.

Interação com o Sistema de Combate

Os modificadores de alcance são aplicados durante o ataque.

Fluxo simplificado:

```
Ataque
↓
Aplicar modificador de distância
↓
Resolver defesa
↓
Aplicar modelo de dano
```

Isso mantém o sistema consistente com:

- [Ataque e Defesa](#)
- [Modelo de Dano](#)

Filosofia de Design

O sistema de alcance do RGB segue três princípios:

- **simplicidade** — poucos modificadores fáceis de lembrar
- **consistência** — mesmo modelo para todas as armas
- **equilíbrio tático** — armas diferentes possuem alcances distintos

Isso permite que o posicionamento tenha impacto real no combate.

Veja Também

- [Armas de Fogo](#)
- [Penetração](#)
- [Modelo de Dano](#)

← [Voltar para Readme](#)